

Dynamic Minimum Bichromatic Separating Circle

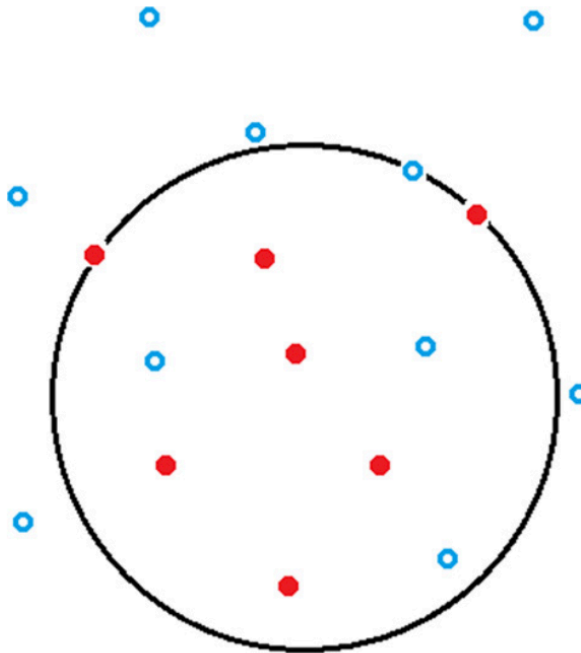
(DMBSC) [1]

Minimum Bichromatic Separating Circle (MBSC) [2]

La base para DMBSC

Minimum Bichromatic Separating Circle (MBSC)

Dado un conjunto R de puntos rojos con $|R| = n$, y un conjunto B de puntos azules con $|B| = m$, buscamos el círculo que contenga todos los puntos rojos, la menor cantidad de puntos azules, y que tenga el menor radio.



- El círculo buscado es llamado *Minimum Separating Circle*, y se denota por $C_B(R)$ o simplemente como *MBSC*.
- La solución no siempre es única.
- Se asume que los puntos en $R \cup B$ están en posición general: no hay tres puntos colineales ni cuatro puntos cocirculares.

Aplicaciones

Usado en ámbitos militares:

- Se busca bombardear una zona, donde los enemigos están marcados de color rojo y los aliados están marcados de color azul.

Al encontrar el MBSC se reduce el gasto y daños provocados.

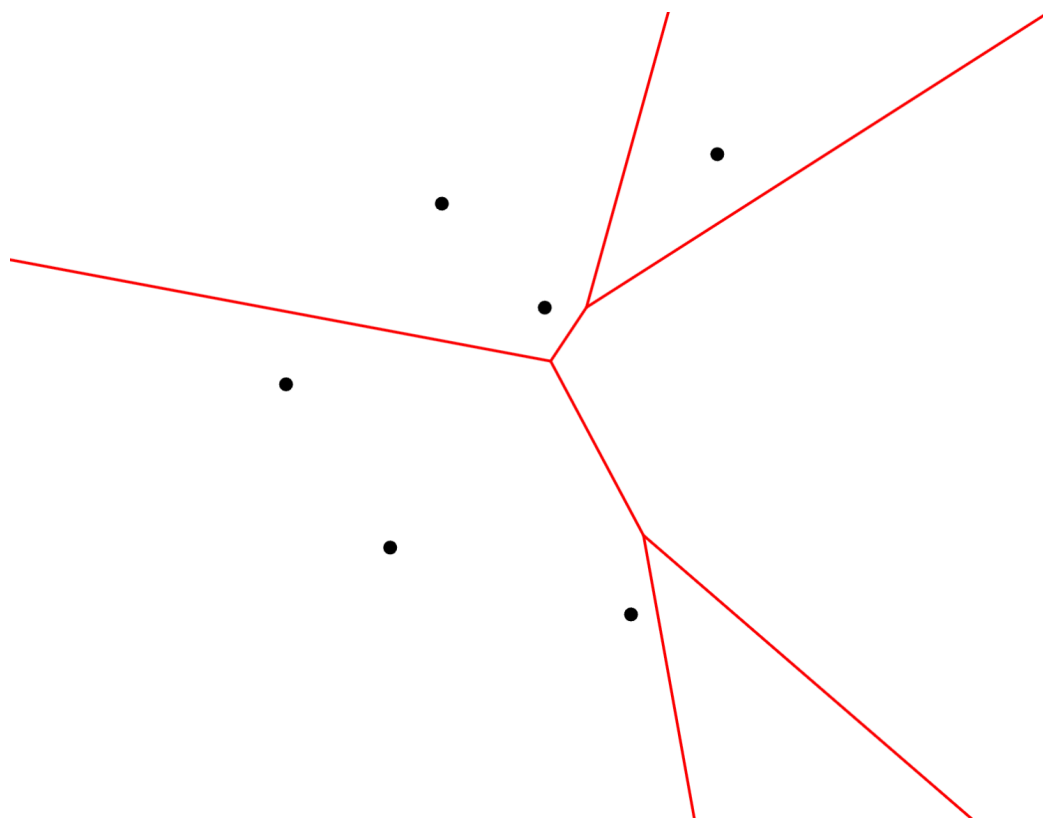
- Para comunicaciones inalámbricas entre diferentes grupos aliados (puntos rojos), mientras se trata de minimizar intercepciones por grupos enemigos (puntos azules).

Conceptos Básicos

- Farthest-point Voronoi Diagram (FVD)
- Minimum Enclosing Circle (MEC)
- Minimum Separating Circle (MSC)
- Convex Hull (CH)

Farthest-point Voronoi Diagram (FVD)

Denotado por $FVD(R)$. Crea una partición del plano en regiones tales que los puntos en una misma región tienen el mismo punto en R como su punto más lejano.



Observación 1.FVD

Un punto tomado sobre un arista del FVD está asociado a dos puntos lejanos.

Observación 2.FVD

Un punto tomado en un vértice del FVD está asociado a tres (o más) puntos lejanos.

Observación 3.FVD

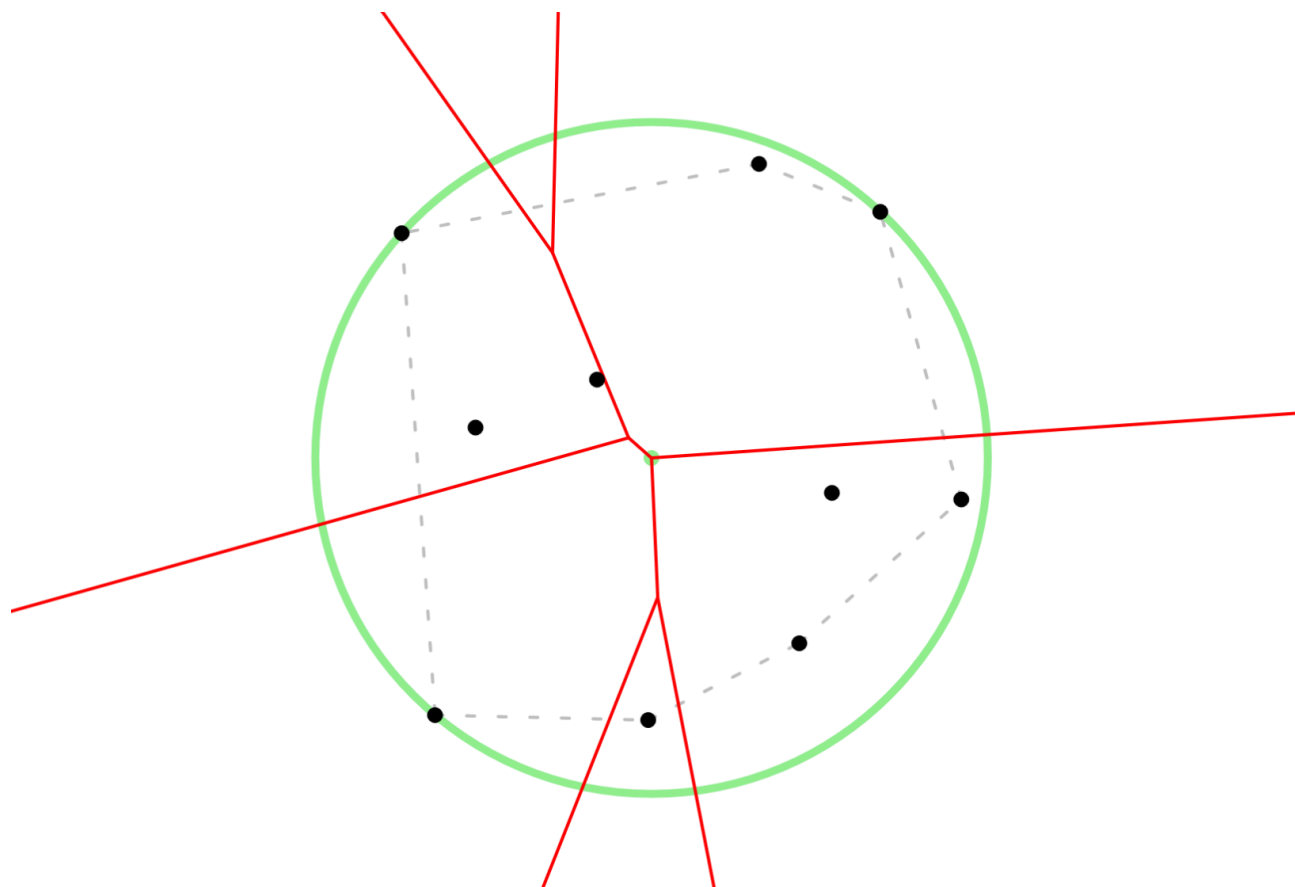
Puede construirse en $O(n)$ a partir del Diagrama de Voronoi Standard (SVD), o en $O(n \log n)$ desde 0.

Observación 4.FVD

Un punto sobre un vértice está sobre una arista.

Minimum Enclosing Circle (MEC)

Circulo de menor radio que encierra todos los puntos de R . Ignora B .



Observación 1.MEC

El MEC tiene su centro c en una arista del $FVD(R)$.

Demostración:

El centro c solo puede estar en dos posiciones:

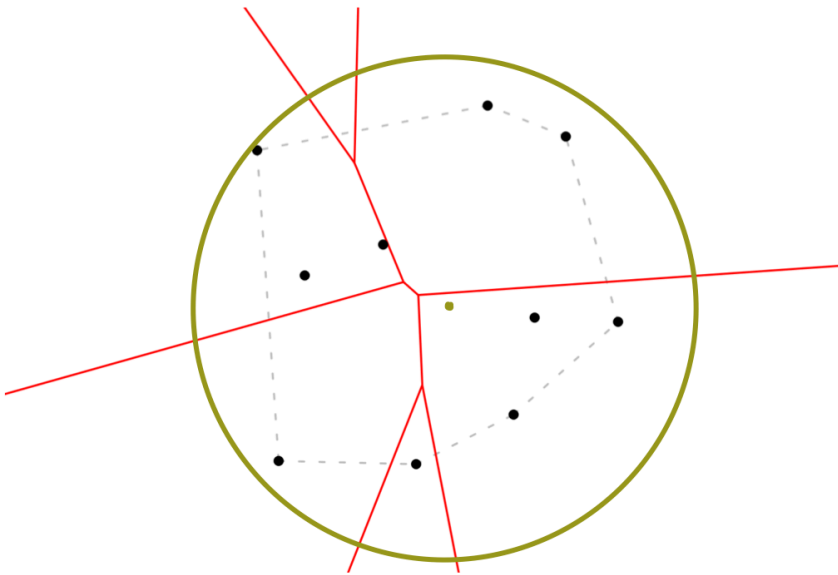
1. Una región del $FVD(R)$:

Entonces, c tiene un solo punto lejano, digamos p_k , tal que $d(c, p_k) > d(c, p_i), \forall p_i \in R$, con $p_k \neq p_i$. Si movemos el c en línea recta hacia p_k , estaríamos consiguiendo un círculo más pequeño —una contradicción.

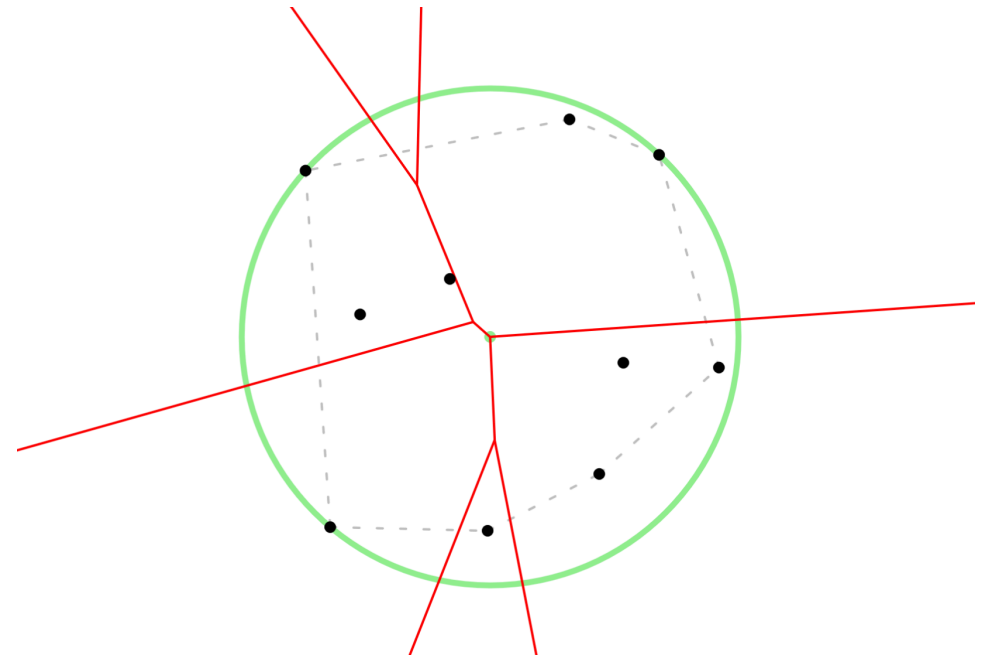
2. Sobre una arista del $FVD(R)$.

Entonces, c está asociado a dos o más puntos lejanos de R .
Si tratamos de mover a c hacia cualquiera de dichos puntos,
uno de ellos quedaría fuera del círculo —rompe la
factibilidad. ■

Centro en una región.

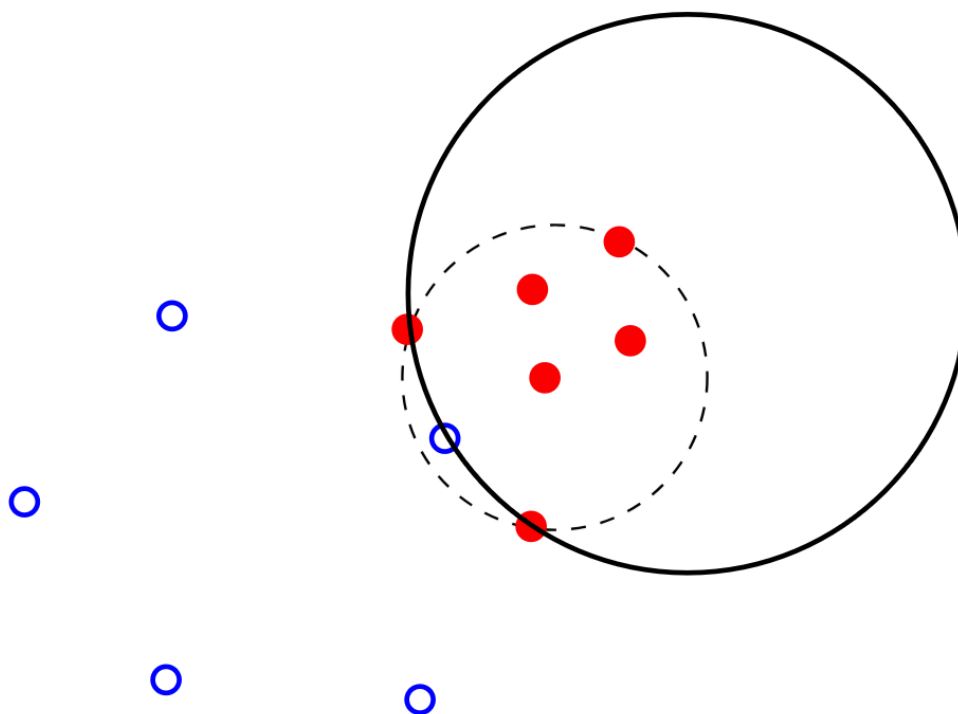


Centro en una arista.



Minimum Separating Circle (MSC/MBSC)

Círculo que encierra todos los puntos de R , minimiza la cantidad de puntos azules, y minimiza el radio.



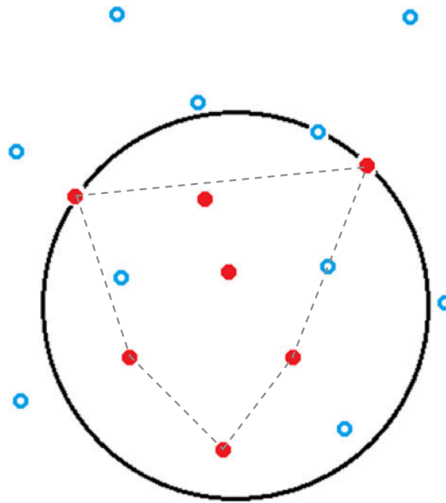
En general, el MBSC y el MEC son dos círculos diferentes.

Convex Hull (CH)

Calculamos el $CH(R)$ en $O(n \log n)$.

Observación 1.CH

Todos los puntos dentro del $CH(R)$ no aportan nada al MBSC.



Dichos puntos son descartados.

Descartar Puntos en B : Tomamos cada punto y evaluamos si se encuentra dentro del $CH(R)$: $O(m \log n)$

Descartar Puntos en R : Consideramos solo los puntos que pertenecen al $CH(R)$.

Observación 2.CH

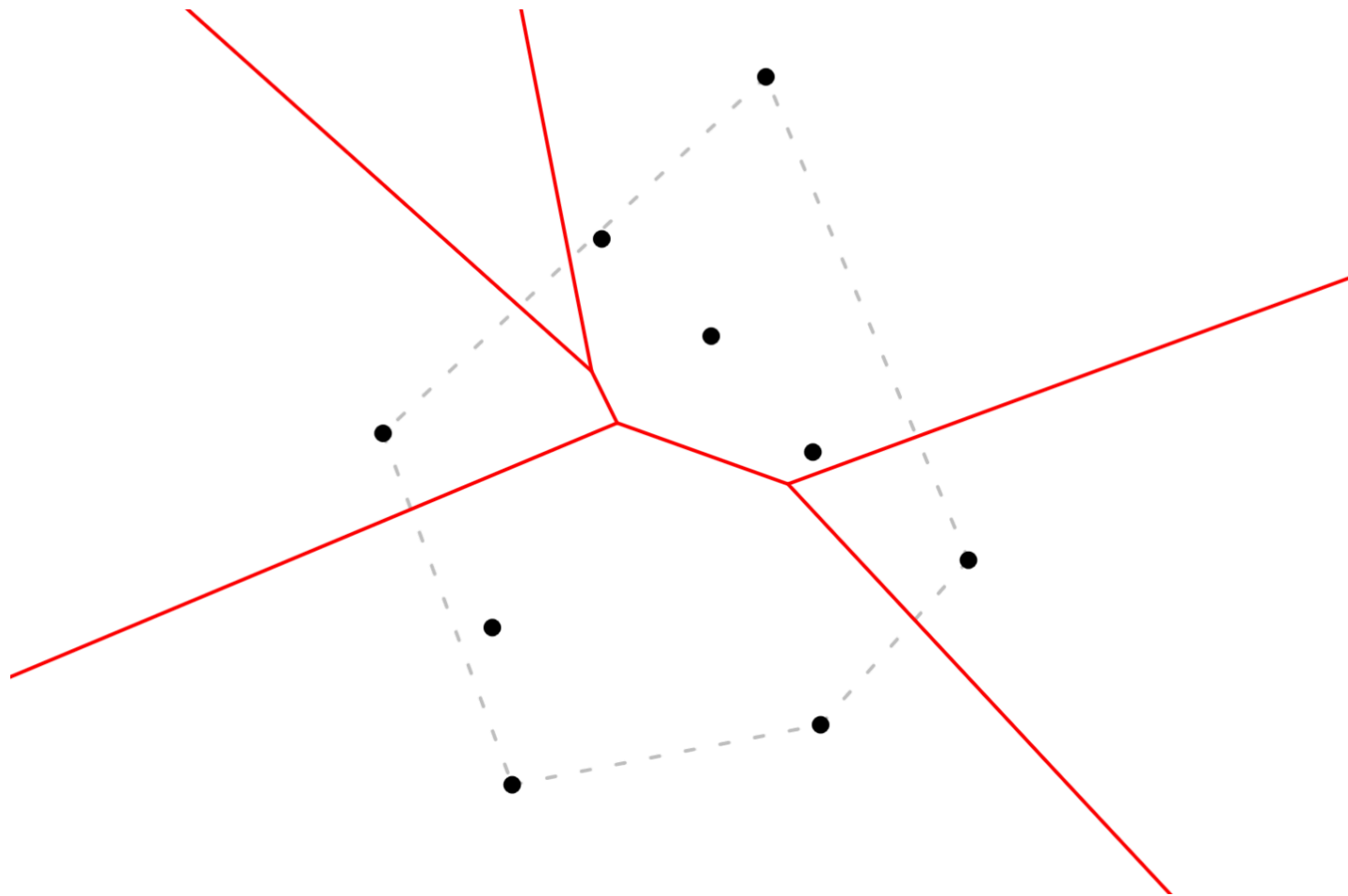
A partir de ahora, se asume que no hay punto rojo que no forme el $CH(R)$, y que no hay punto azul dentro del $CH(R)$.

Observación 3.CH

Cada punto del $CH(R)$ tiene una región asociada en $FVD(R)$.

Observación 4.CH

Por cada punto del $CH(R)$ hay una arista no acotada/infinita.



Lemas para MBSC

Lema 1.MBSC

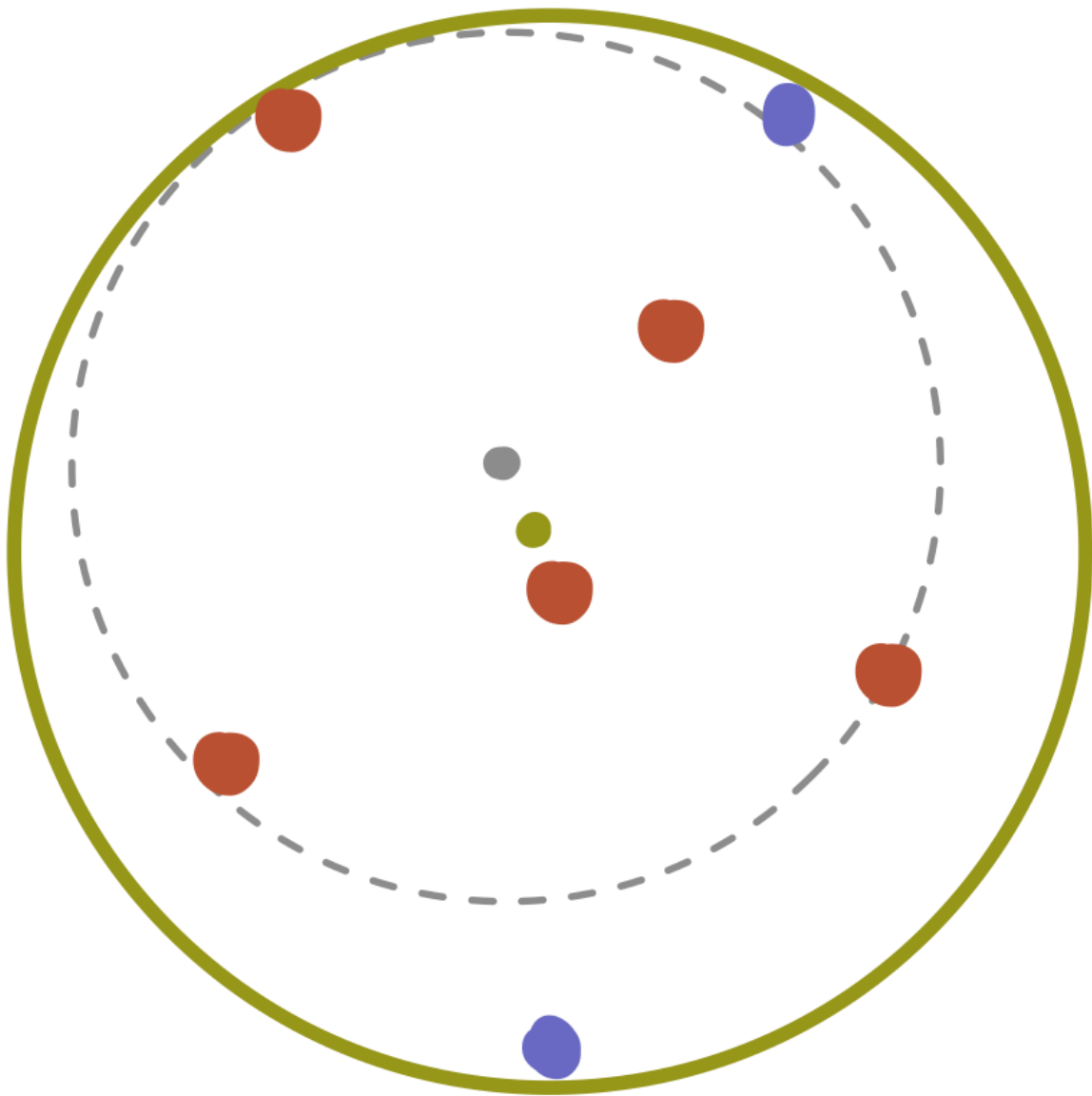
El MBSC debe pasar por al menos dos puntos de R .

Demostración:

Supongamos lo contrario. Entonces, el círculo debe estar definido por:

- Un punto rojo y uno azul, o
- Tres puntos azules.

En ambos casos podemos fijar uno de los puntos, digamos p_i , y mover el centro del círculo hacia p_i una distancia ε pequeña, tal que sigue conteniendo a todos los puntos de R —es una contradicción. ■



Lema 2.MBSC

Hay n MBSC's en el peor de los casos.

Demostración:

Supongamos que existen $l > n$ MBSC.

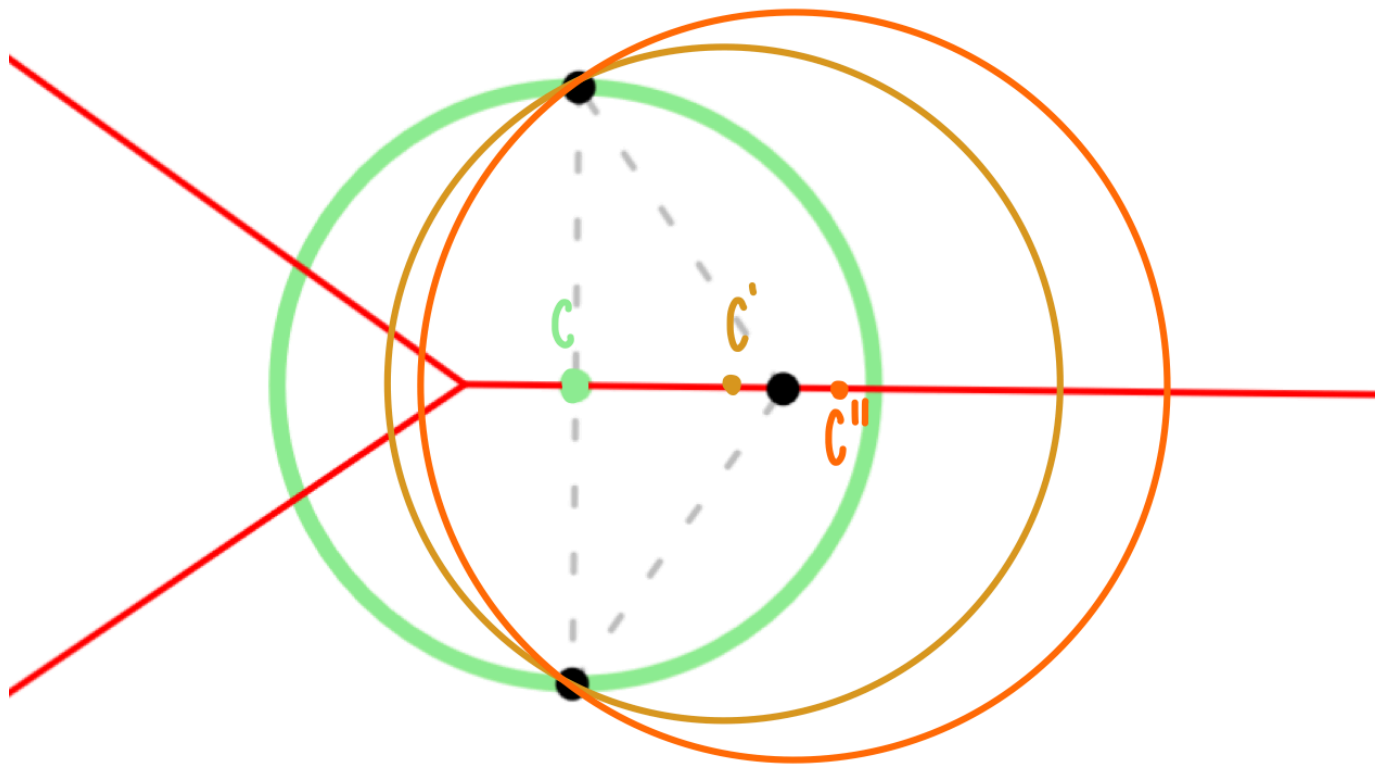
Por *Observación 1.MEC*, el centro c del MEC está en una arista de $FVD(R)$. Esto nos define una dirección de movimiento: hacia c o en dirección opuesta a c .

Como el MEC es mínimo, cualquier círculo que contenga a todo R con centro c' que se aleje por una arista de c tendrá un radio mayor.

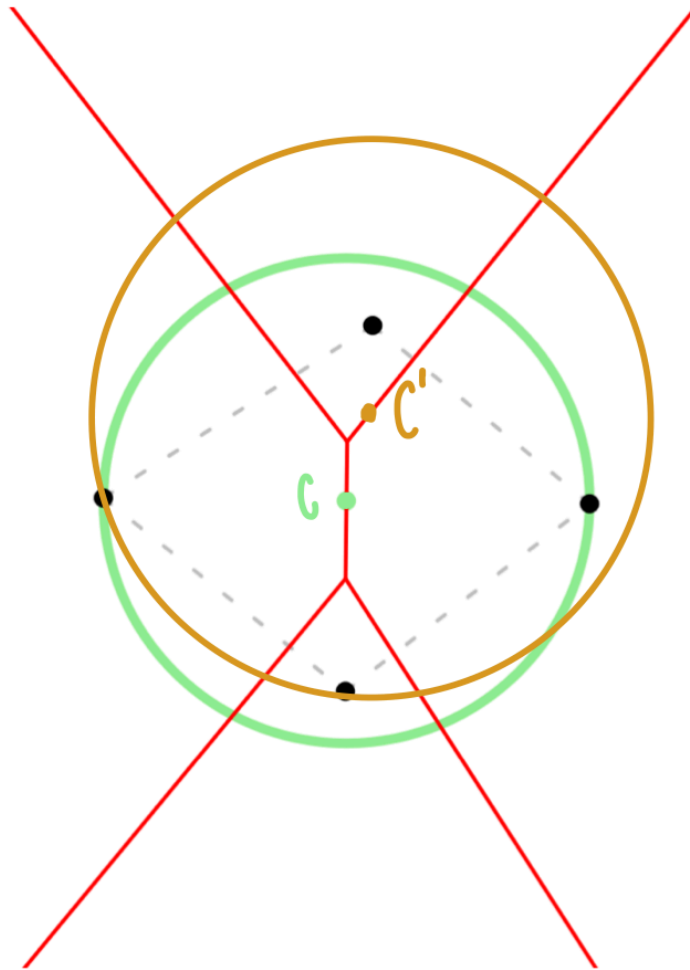
Consideramos $c', c'' \in e$ centros diferentes de dos MBSC's.

- Si $d(c', c) > d(c'', c)$, el MBSC centrado en c' no era mínimo.
- Si $d(c'', c) > d(c', c)$, el MBSC centrado en c'' no era mínimo.

Así que solo puede haber un MBSC por arista infinita. Por la *Observación 3.CH*, tenemos n posibles MBSC usando las ramas infinitas.

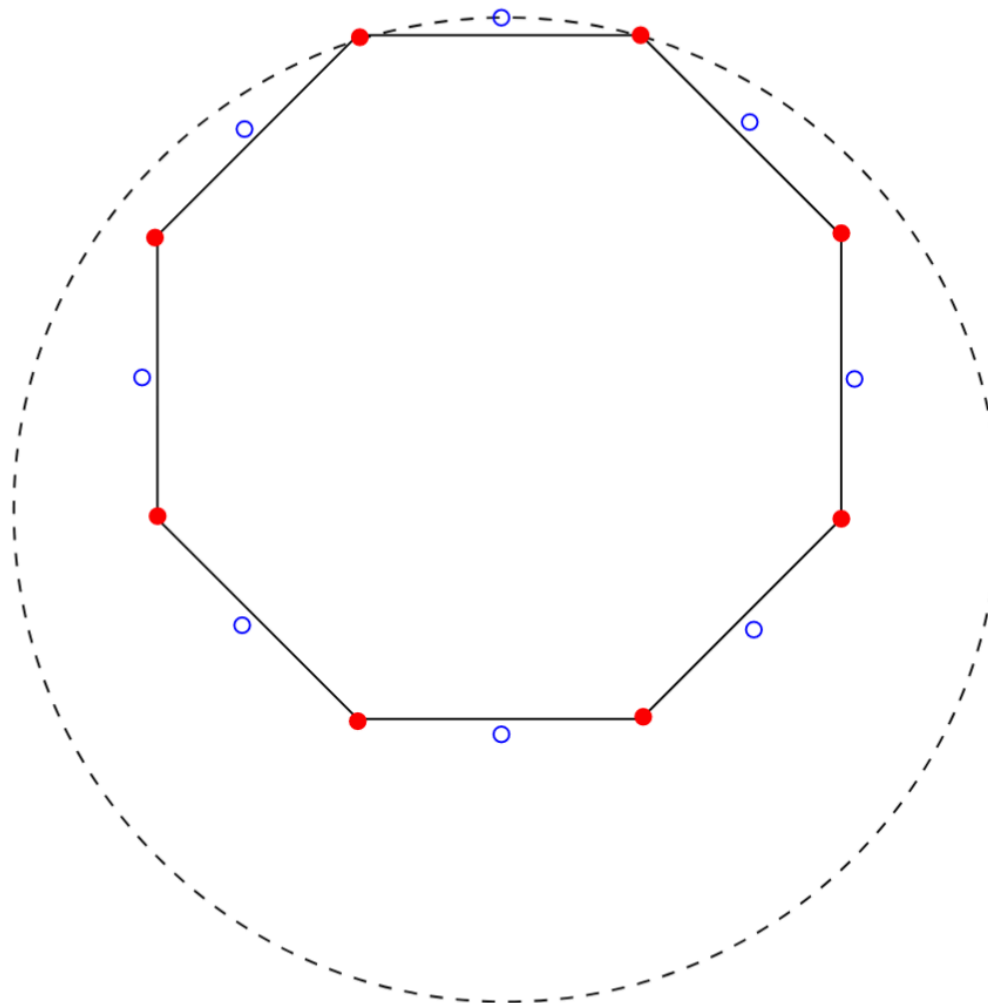


Si el centro de c' de un MBSC cae en una arista interna e , el razonamiento es el mismo para los círculos con centro en e , pero nos reduce el espacio de búsqueda al descartar todas las aristas infinitas que se bifurcan de la rama interna.



Entonces, como limite superior, tenemos n MBSC's.

Es posible formar un caso donde el número de MBSC posibles es exactamente n .



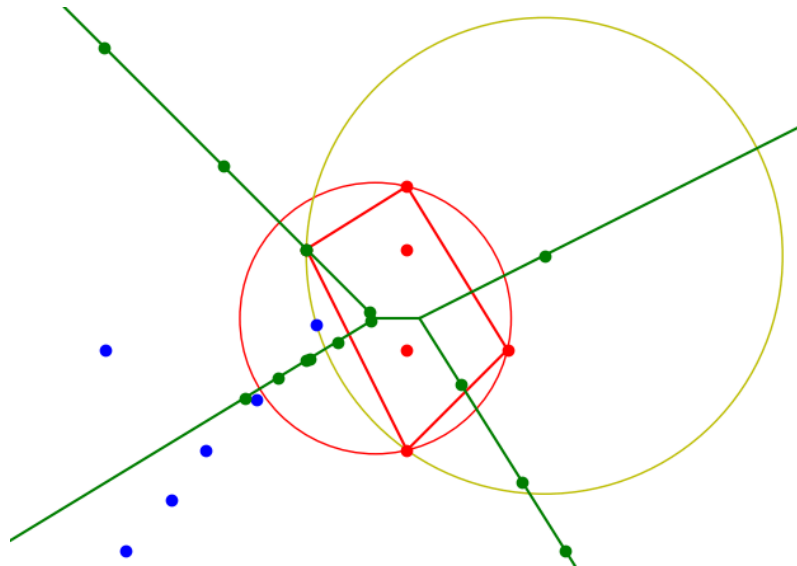
Lema 3.MBSC

El centro de un MBSC cae en una arista del $FVD(R)$.

Demostración:

Por el lema 1.MBSC, el MBSC toca al menos dos puntos de R .

Para que un círculo toque dos puntos (o más) lejanos, por definición, su centro debe estar en una arista (vértice). ■



Algoritmo Ingenuo para Encontrar MBSC

Hay $m\binom{n}{2}$ círculos formados con un punto rojo y uno azul.

Si el círculo contiene a R , contamos cuantos puntos azules contiene.

Entonces, el MSC puede ser encontrado en $O(n^2 m(m + n))$. Si $n \approx m$, el algoritmo es cuártico.

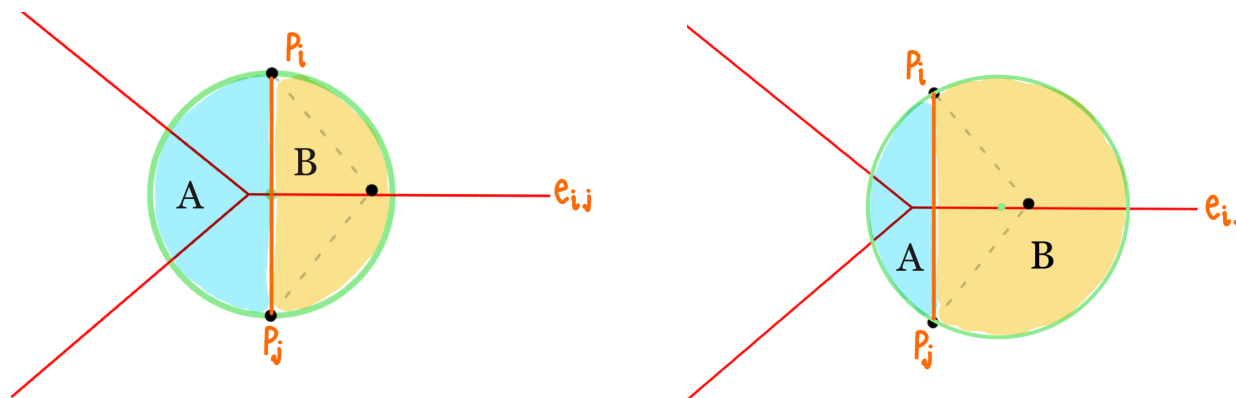
```
mbsc = get_mec_circle() // O(n)
min_bp = get_bp(mbsc) // O(m)
circles = generate_all_circles() // O(m n^2)
for c in circles: // O(m n^2)
    if c.contains(R): // O(n)
        bp = get_bp(c) // O(m)
        if bp < min_bp || (bp == min_bp && rad(c) < rad(mbsc)):
            mbsc = c
            min_bp = bp
return mbsc
```

Algoritmo Eficiente para Encontrar MBSC

Segmento $\overline{p_i p_j}$, y Regiones A y B

Todo círculo con centro sobre $e_{i,j}$ y que pasa por p_i y p_j es dividido en dos mediante $\overline{p_i p_j}$:

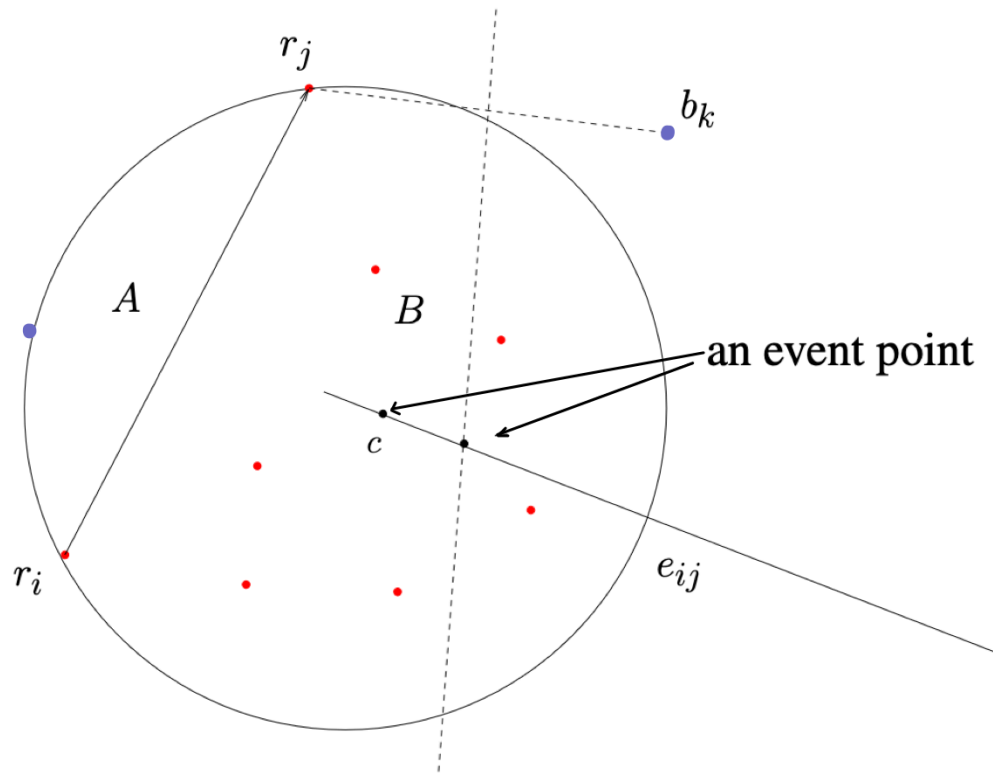
- Región A: Se achica al movernos por $e_{i,j}$.
- Región B: Se agranda al movernos por $e_{i,j}$.



El radio crece o decrece conforme nos desplazamos.

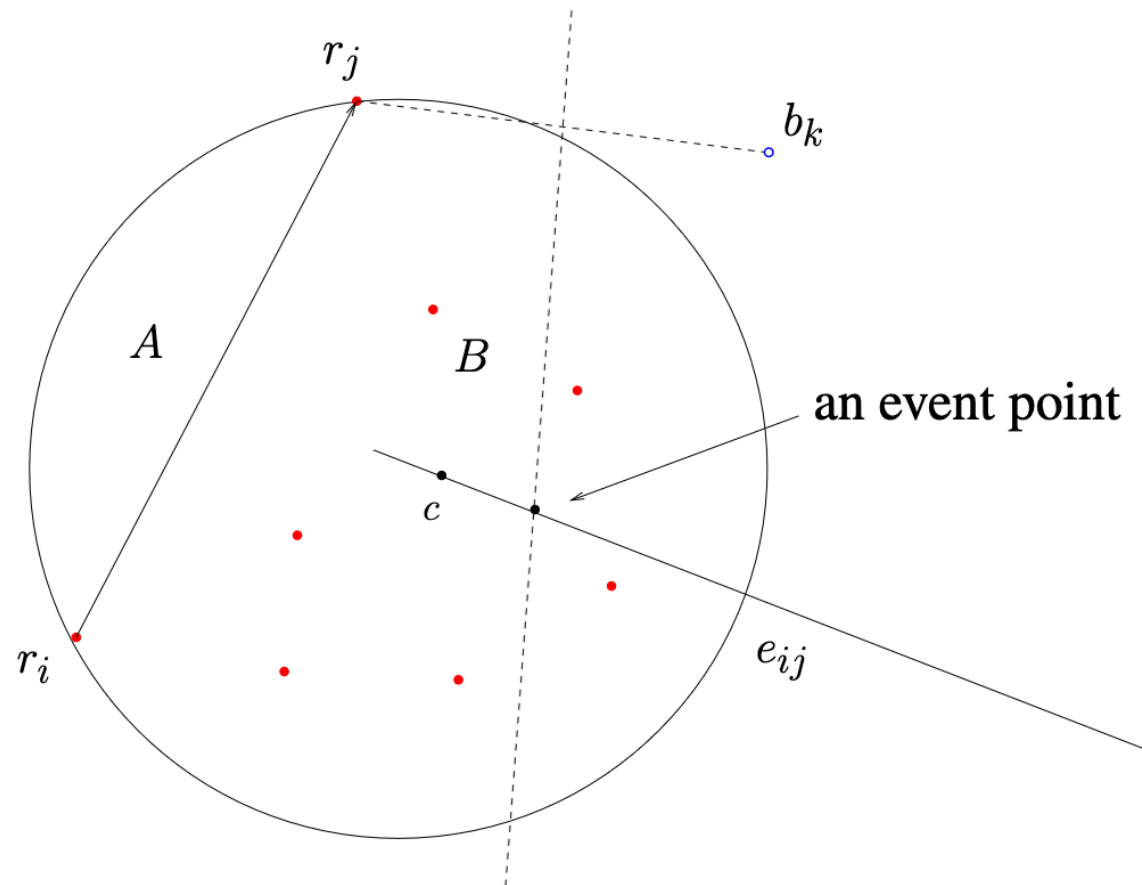
Punto de Evento

Un punto q en una arista $e_{i,j}$ del $FVD(R)$ es un punto de evento entrante (saliente) si cuando el círculo centrado en q incluye (excluye) a un punto azul.



Cálculo de Punto de Evento

Cada punto de evento (entrante o saliente) se calcula en $O(1)$ al encontrar la intersección entre $e_{i,j}$ y la mediatriz de $\overline{r_j b_k}$, con b_k un punto azul. Para cada arista hay m posibles puntos de eventos — $O(m)$



Teorema 1

El MBSC se puede encontrar en $O(nm \log m + n \log n)$.

Demostración:

Generamos el FVD(R) en $O(n \log n)$.

Para cada arista $e_{i,j}$ del FVD(R):

Creamos el círculo con centro c que pasa por p_i y p_j , tal que c es el punto más cercano a $\overline{p_i p_j}$.

Calculamos en $O(m)$ los puntos de evento de $e_{i,j}$, y los ordenamos en $O(m \log m)$ respecto a su distancia a c .

En una variable *blue_points*, contamos en $O(m)$ cuantos puntos azules hay en el interior.

Mientras haya puntos de eventos para $e_{i,j}$:

Si el evento es entrante, $blue_points += 1$. Si el evento es saliente, $blue_points -= 1$.

Si $blue_points$ es menor, el circulo actual es un candidato a ser el MBSC. Si hay empate en cantidad de puntos azules, el mejor candidato es el de menor radio. ■

```
mbsc = Null
min_bp = infinity

def update_best_candidate(c, bp):
    if mbsc == Null ||
        bp < min_bp ||
        (bp == min_bp && rad(c) < rad(mbsc)):
        min_bp = bp
        mbsc = c
```

```

def get_mbsc():
    FVD = generate_fvd(R) //  $O(n \log n)$ 
    for e in edges(FVD): //  $O(n)$ 
        c = min_circle_between_points(ri, rj) //  $O(1)$ 
        bp = count_blue_points(c) //  $O(m)$ 

        update_best_candidate(c, bp) //  $O(1)$ 
        if bp == 0:
            continue

        event_points = calculate_event_points(e) //  $O(m)$ 
        sort(event_points) //  $O(m \log m)$ 

        for event_p in event_points: //  $O(m)$ 
            update_bp(event_p)
            update_best_candidate(event_p, bp)

            if bp == 0:
                break

    return mbsc

```

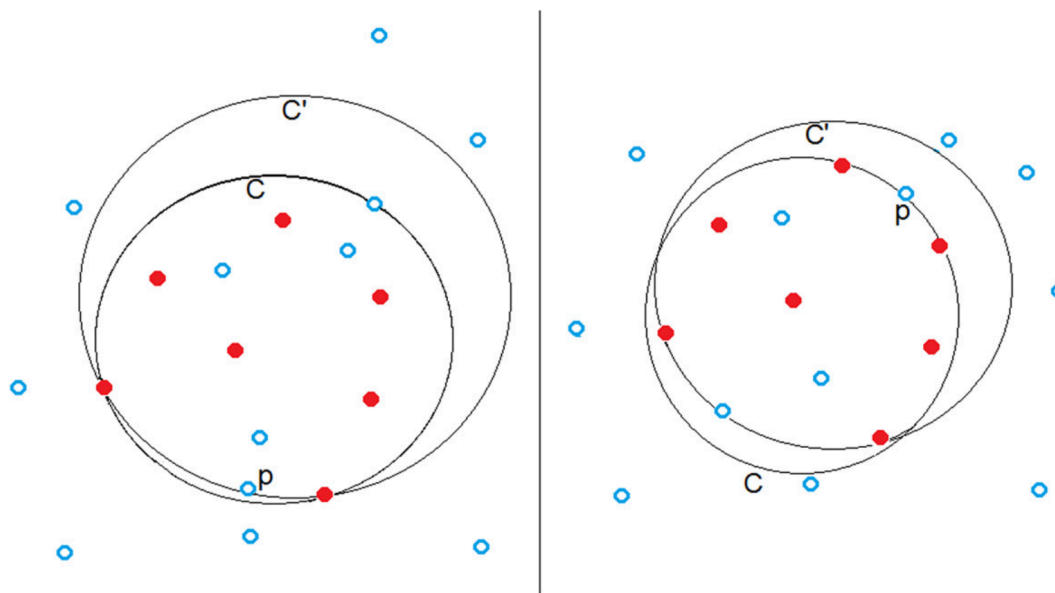
Resumen del MBSC

- Es el caso estático de Dynamic Bichromatic Minimum Separating Circle.
- Usa un barrido sobre las aristas del $FVD(R)$
- La solución no siempre es única.
- No es lo mismo que el MEC.
- Preprocesa los inputs para descartar puntos dentro del $CH(R)$:
 $O(n \log n + m \log n)$
- Las complejidades son:
 - Tiempo: $O(nm \log m + n \log n)$
 - Espacio: $O(n + m)$

Dynamic Minimum Bichromatic Separating Circle (DMBSC)

Dynamic Minimum Bichromatic Separating Circle

Dado un conjunto R de puntos rojos con $|R| = n$, y un conjunto B de puntos azules con $|B| = m$, buscamos el MBSC bajo inserciones y eliminaciones dinámicas de puntos en R y en B .



A la izquierda se inserta p . A la derecha se elimina p . En ambos casos la solución óptima cambia y es necesario re-calcularla.

Aplicación

Mismo ejemplo que antes, pero considerando actualizaciones:

Si durante la misión se descubren nuevos enemigos (aliados), se quisiera recalcular el radio de circulo para eliminarlos (no dañarlos), pero sin tener que recurrir al algoritmo del MBSC.

Preprocesado

En este caso solo se consideran inserciones y eliminaciones de puntos de B , mientras que el conjunto R está fijo.

Observación 1

Si R se mantiene fijo, el $FVD(R)$ no cambia.

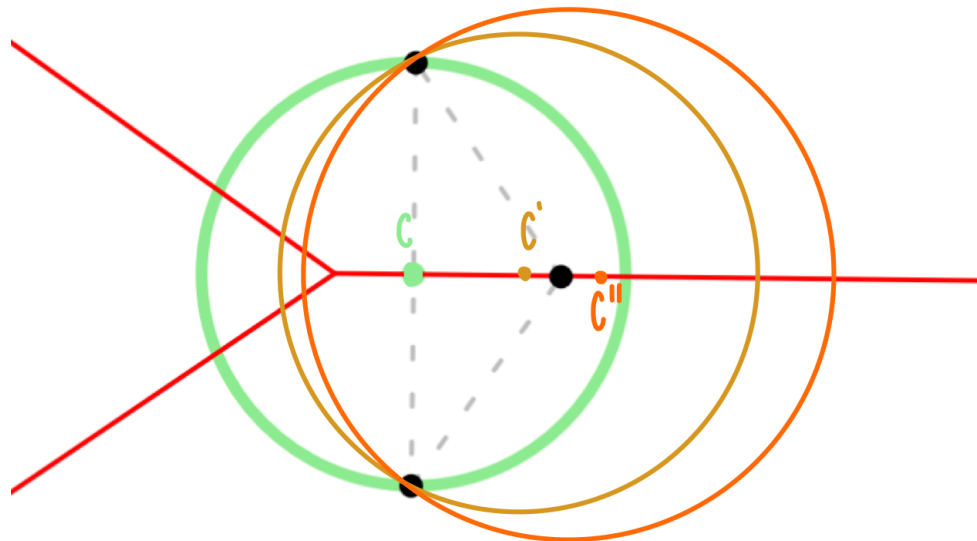
Se inicia ejecutando el algoritmo del MBSC en $O(nm \log m + n \log n)$.

Los MBSC encontrados por el algoritmo estático son guardados en un conjunto M .

Observaciones y Conceptos Necesarios

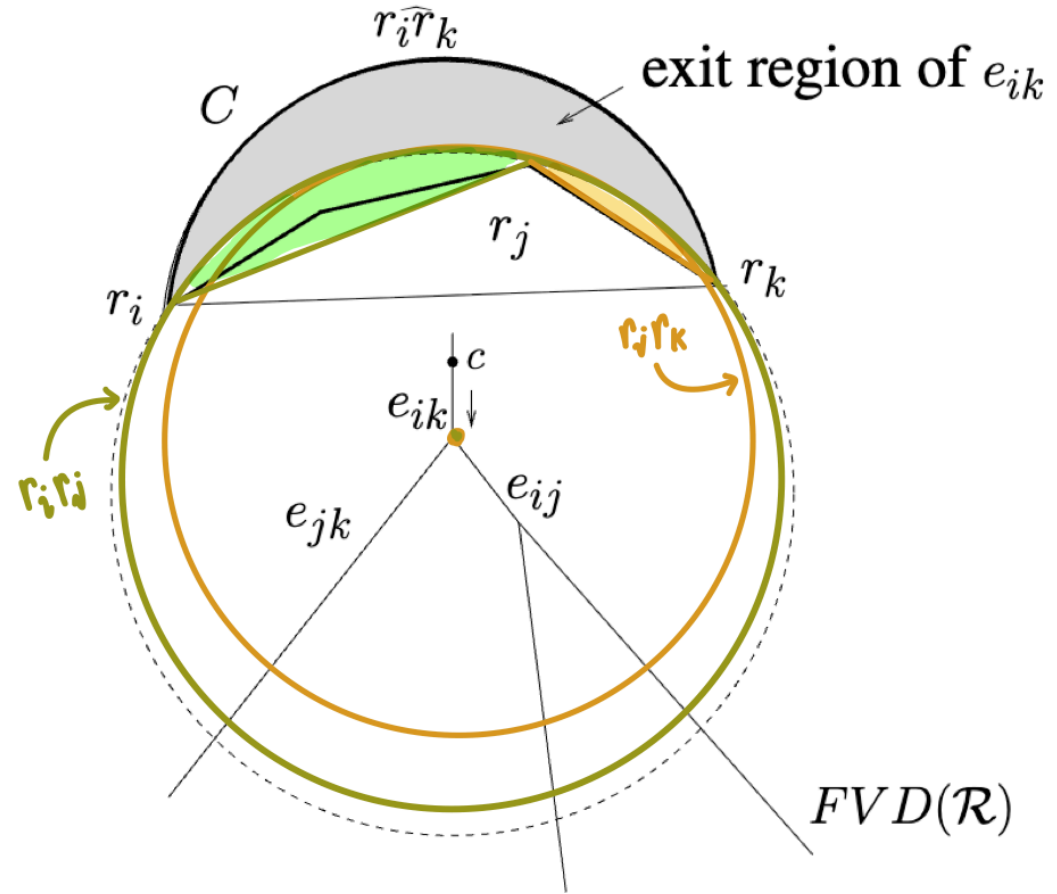
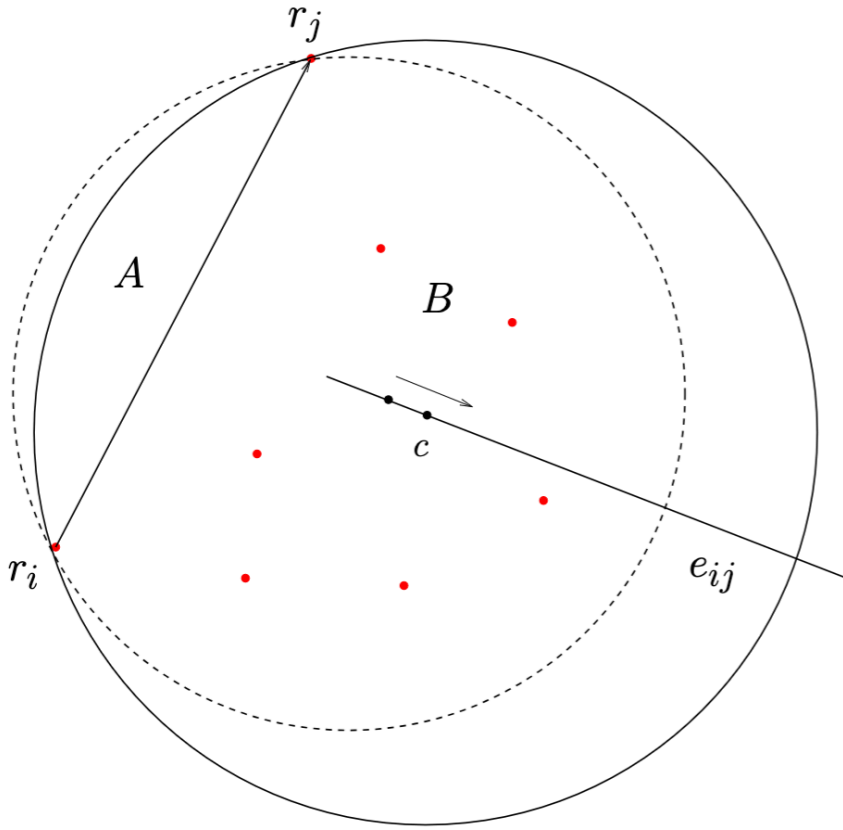
Definición 1: $q < r$

Dada $e_{i,j}$ la arista de $FVD(R)$ que está definida por los puntos $p_i, p_j \in R$.
Para cualquier $q, r \in e_{i,j}$ puntos, decimos que q está a la izquierda de r ($q < r$), si el círculo con centro en q y definido por p_i y p_j tiene un radio menor que el círculo centrado en r y definido por p_i y p_j .



Observación 2

Un punto en B tiene un punto de evento saliente en a lo más una arista de $FVD(R)$. Entonces, hay $O(m)$ puntos de evento salientes en todo el $FVD(R)$.



Observación 3

Un MBSC es el MEC, o está centrado en un punto de evento saliente.

Consideramos al MEC debido a que es el círculo con menor radio que encierra a todo R .

Cuando pasamos por un punto de evento, el número de puntos azules dentro de él disminuye. Para puntos de evento entrantes, es lo contrario.

Observación 4

Consideramos el centro del MEC como un punto de evento saliente.

Por simplicidad para estandarizar los criterios de evaluación al busca un candidato a un MBSC.

Notaciones

1. $C(q)$: Círculo centrado en el punto q sobre la arista e . Pasa por los dos puntos rojos que definen e .
2. $\text{rad}(C)$ y $m(C)$: Radio del círculo C y la cantidad de puntos azules que tiene adentro, respectivamente.
3. $\text{rad}(q)$: Radio del círculo $C(q)$ para un punto de evento q .
4. m_q : Número de puntos azules dentro de $C(q)$.
5. q_e^* : Punto de evento saliente, tal que el círculo centrado en él es el mejor candidato local en la arista e a ser MBSC.
6. k : Número actual de puntos azules en el MBSC .
7. p : Punto a ser insertado o eliminado, y lo llamamos query point.

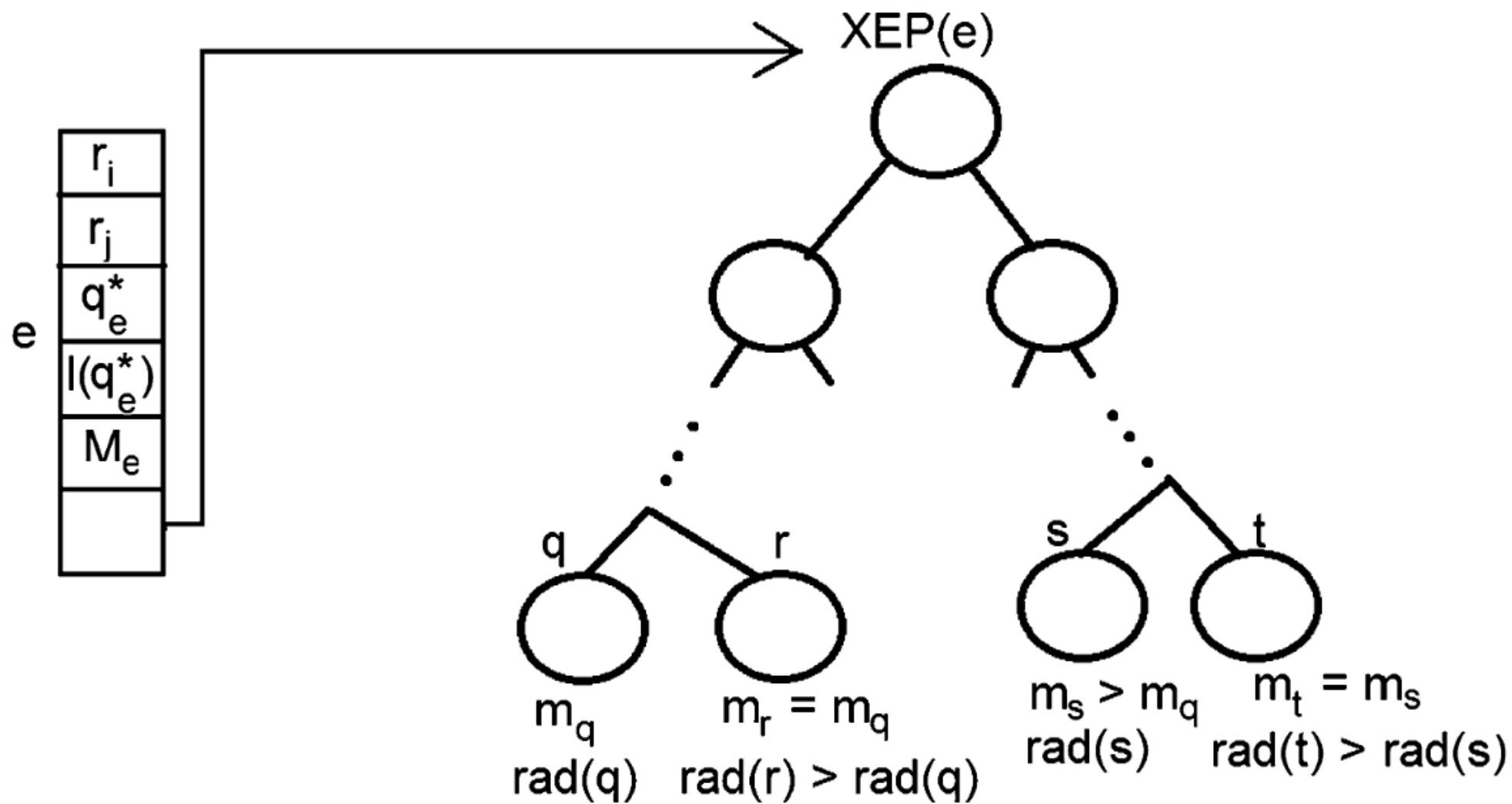
8. $l(s)$: Punto de evento $q \in e$ tal que $q < r$ y $m_q = m_{q_e^*} + 1$.
9. $r(s)$: Punto de evento $q \in e$ tal que $s \leq q$ y $m_q = m_{q_e^*}$.

Modificación del algoritmo MBSC

Modificamos la salida. En lugar de regresar un sólo MBSC, regresamos:

Un array A , tal que para cada $e \in \text{FVD}(R)$, $A[e]$ contiene:

1. r_i, r_j : Puntos que definen la arista e .
2. q_e^* : Mejor candidato local de e a ser MBSC.
3. $l(q_e^*)$: Segundo mejor candidato de e a ser MBSC.
4. M_e : Circulo con centro en q_e^*
5. Puntero a un árbol binario de búsqueda balanceado $\text{XEP}(e)$ con todos los puntos de eventos salientes en la arista e , indexados por m_q (número de puntos azules) y por $\text{rad}(q)$.



Observación 5

Ninguno de estos cambios afecta las complejidades en tiempo y espacio del algoritmo MBSC.

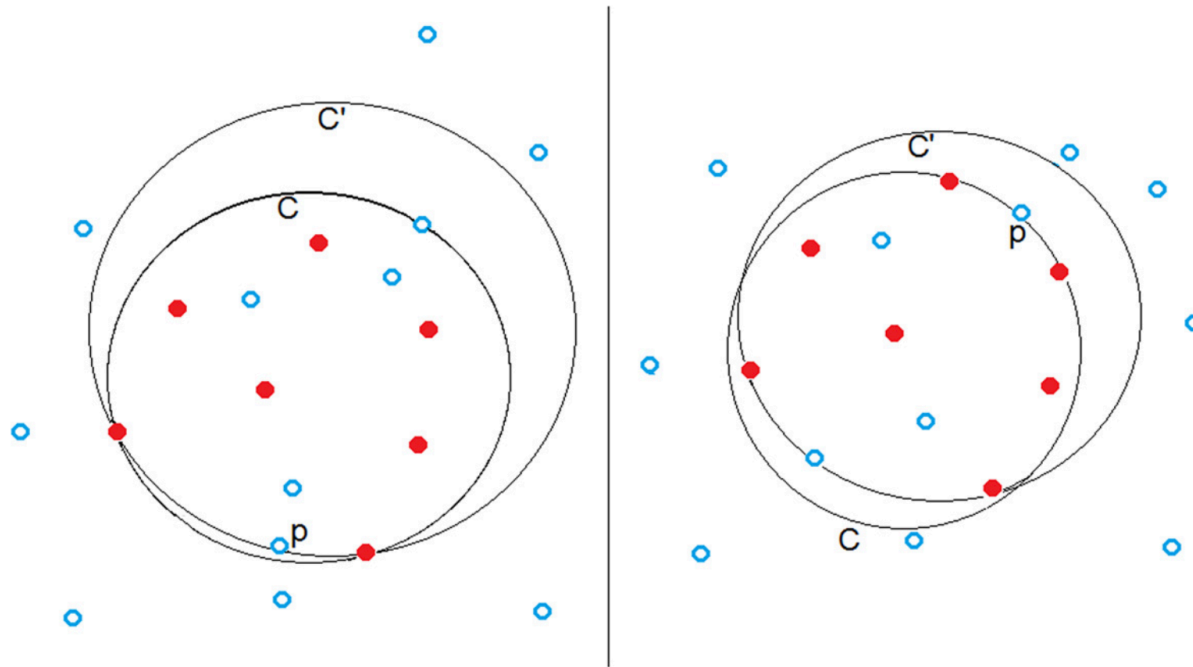
Demostración: Cada elemento en $A[e]$ es calculado durante el algoritmo MBSC. Por la *Observación 2*, tenemos $O(m)$ puntos de evento salientes, así que el espacio total de todos los $XEP(e_i)$ con $e_i \in FVD(R)$ es $O(n + m)$. Las construcciones se hacen en $O(m \log m)$.

Los nodos son los puntos de eventos salientes calculados en el algoritmo MBSC.

Entonces, se conservan las complejidades de $O(n \log n + mn \log m)$ y $O(n + m)$ en tiempo y espacio, respectivamente. ■

Método Unificado para Inserciones y Eliminaciones de Puntos Azules

Eficiente para inserciones y eliminaciones de puntos azules.



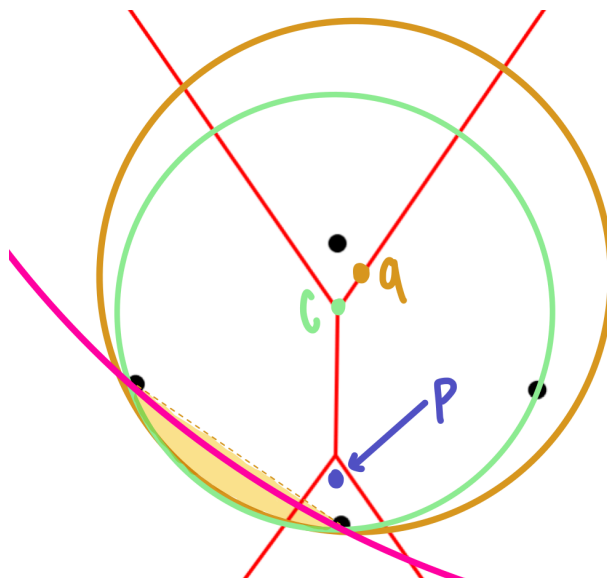
C la solución previa a la inserción (eliminación) de p . C' la nueva solución.

Insertar punto (Insert-Blue-Query(A, k, p))

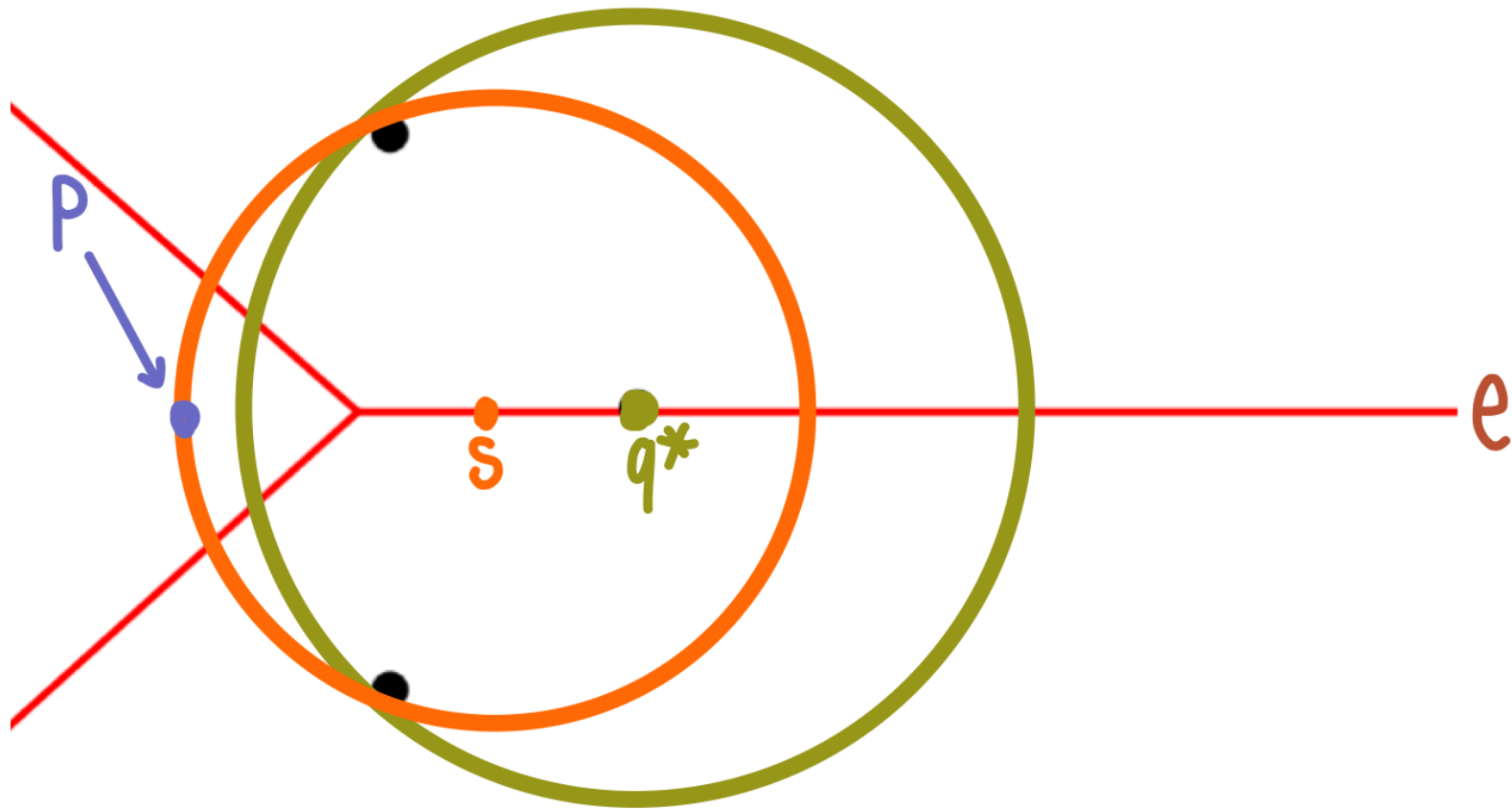
Si $p \in CH(R)$, entonces el MBSC actual sigue siendo el óptimo.

Si $p \notin CH(R)$, consideramos s el punto de evento generado por p (calculado en $O(1)$). Por cada $e \in FVD(R)$ tenemos 5 casos:

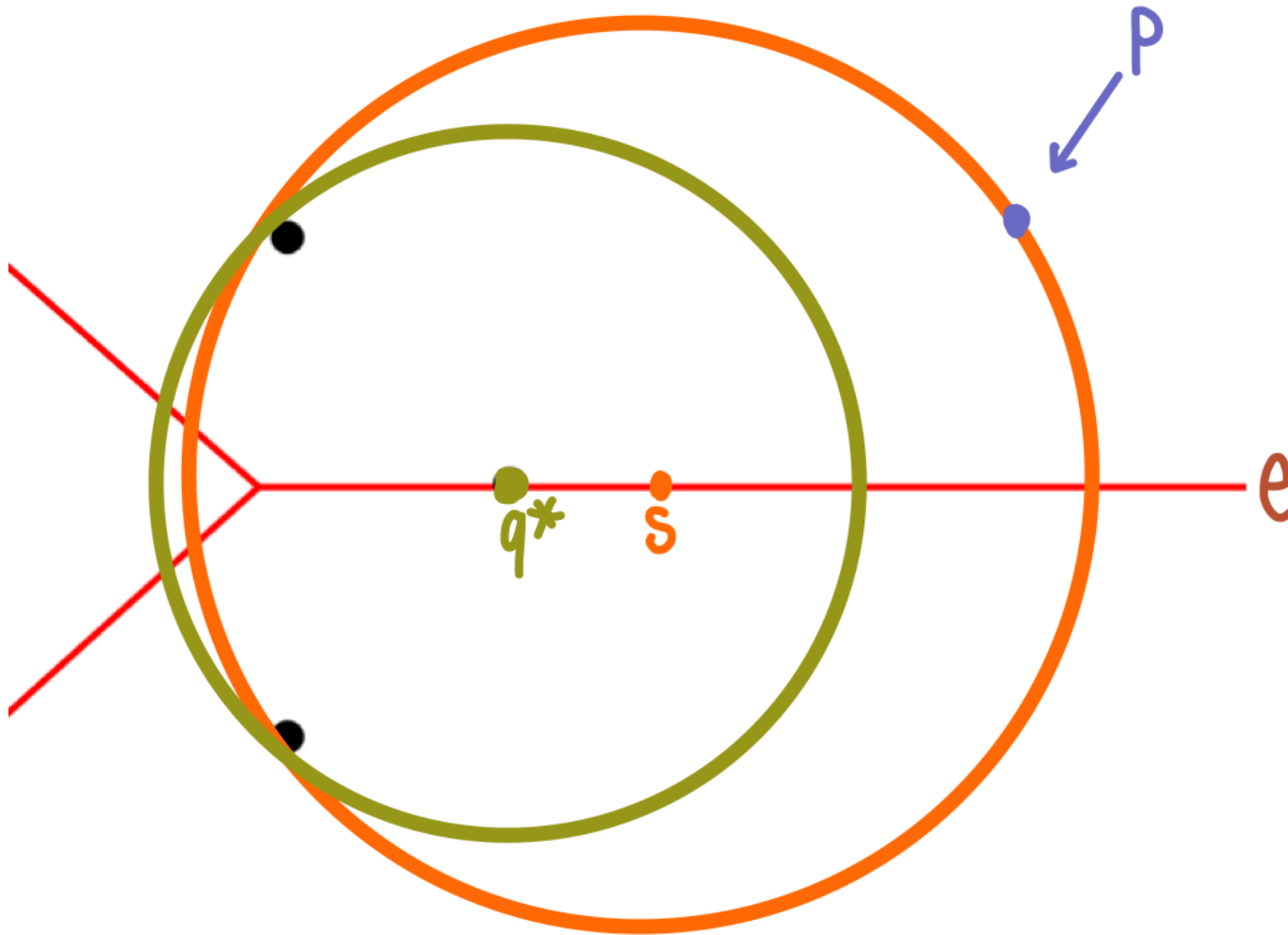
1. s no es un punto de evento en e : El MBSC actual es el óptimo. Si los círculos en e contienen a p , sumamos un punto azul.



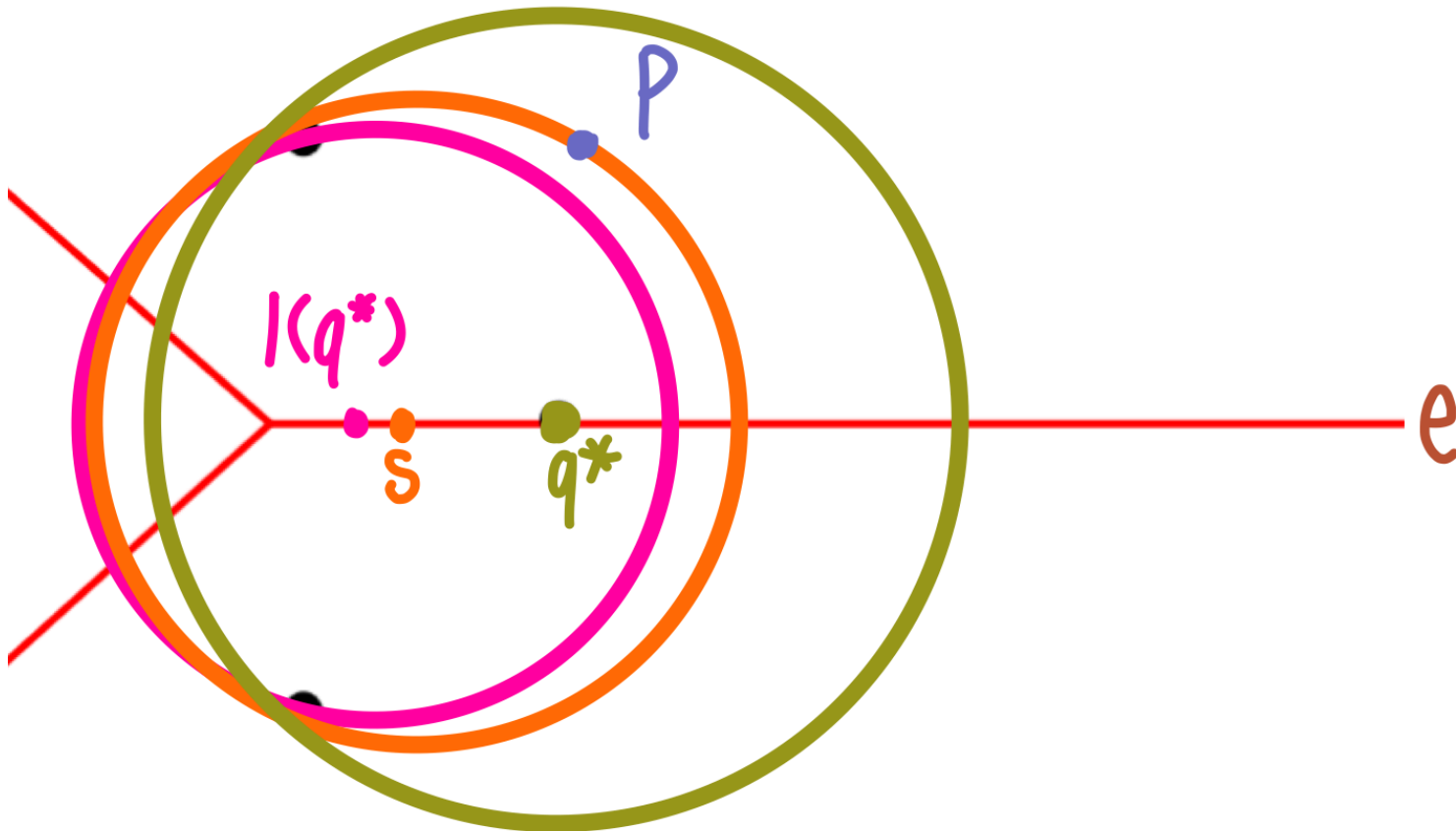
2. $s < q_e^*$ y s es un punto de evento saliente: s sale antes de que $C(q_e^*)$ es creado. El MBSC óptimo y el número de puntos azules se mantienen igual.



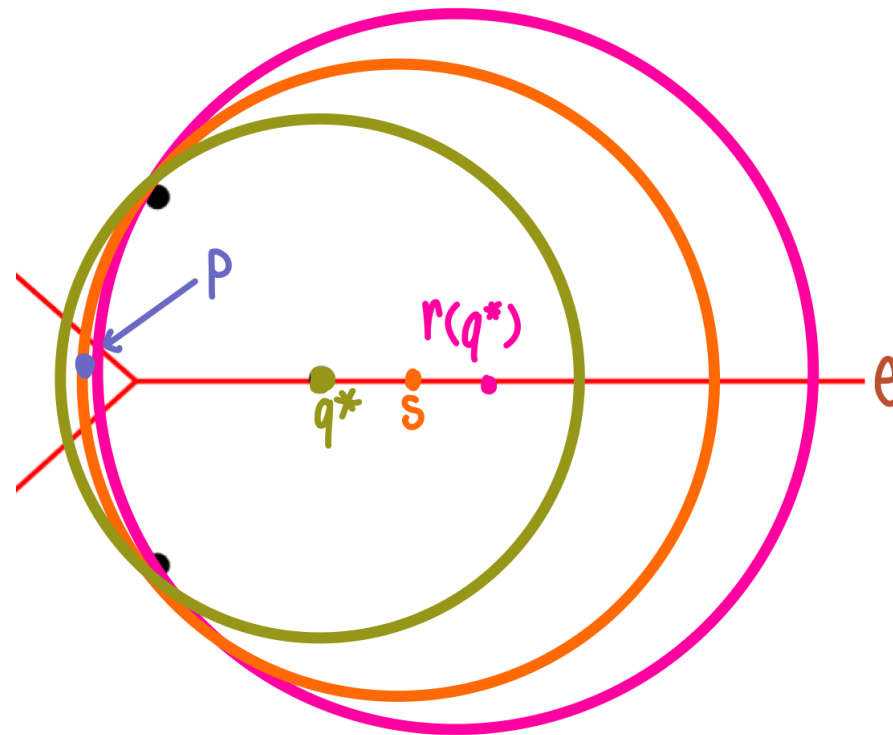
3. $q_e^* < s$ y s es un punto de evento entrante: $C(q_e^*)$ es creado antes de llegar a s . El MSC óptimo y el número de puntos azules se mantienen.



4. $s < q_e^*$ y s es un punto de evento entrante: $C(q_e^*)$ tiene un punto azul más.
- Si $l(q_e^*) < s$ existe, $C(l(q_e^*))$ es el nuevo MBSC con un punto azul menos. En otro caso, el MBSC actual se mantiene y sumamos un punto azul.



5. $q_e^* < s$ y s es un punto de evento saliente: $C(q_e^*)$ tiene un punto azul más.
- Si $r(s) \geq s$ en $\text{XEP}(e)$ existe, $C(r(s))$ es el nuevo MBSC con el mismo número de puntos azules. En otro caso, el MBSC actual se mantiene, y sumamos un punto azul.



Después de procesar las n aristas, seleccionamos el MBSC tal que $m(\text{MBSC})$ y $\text{rad}(\text{MBSC})$ sean los menores.

Lema 1.DMBSC

Dado un nuevo punto p , los MBSC's son reportados en $O(n + \log m)$ usando el algoritmo Insert-Blue-Query.

Demostración:

Busca en las n aristas del $\text{FVD}(R)$ en tiempo $O(n)$.

Por cada $e \in \text{FVD}(R)$, se calcula el punto de evento s de p y su tipo en tiempo $O(1)$.

Los casos 1), 2), 3), y 4) se resuelven en tiempo $O(1)$: Posición de s respecto a q_e^* y tipo de evento. Para 4) usamos $l(q_e^*)$.

Para 5), se busca en tiempo $O(\log m)$ en $XEP(e)$ a $r(s)$. Primero por número de puntos azules y después por radio. Como s es saliente, solo puede ocurrir una vez en todo el diagrama.

Sumando en total $O(n + \log m)$. ■

Actualización de Inserción (Insert-Blue-Update(A, k, p))

Para cada $e \in \text{FVD}(R)$, $s \in e$ el punto de evento correspondiente de p , procedemos según el caso ejecutado en *Insert-Blue-Query*:

1. Si p está contenido en los círculos de e , incrementamos m_q en 1 para todo q evento saliente en $\text{XEP}(e)$.
2. Para todo $q < s$ evento en $\text{XEP}(e)$, incrementamos m_q en 1, y lo relocalizamos en el árbol. Insertamos s en $\text{XEP}(e)$. Re-calculamos $l(q_e^*)$.
3. Para todo $q > s$ evento en $\text{XEP}(e)$, incrementamos m_q en 1, y lo relocalizamos en el árbol.

4. Para todo $q > s$ evento saliente en $\text{XEP}(e)$, incrementamos m_q en 1, y lo relocalizamos en el árbol. Si $l(q_e^*)$ estaba definido, $q_e^* = l(q_e^*)$. Recalculamos $l(q_e^*)$.
5. Para todo $q < s$ evento saliente en $\text{XEP}(e)$, incrementamos m_q en 1, y lo relocalizamos en el árbol. Insertamos s en $\text{XEP}(e)$. Si $r(s)$ estaba definido, $l(q_e^*) = q_e^*$ y $q_e^* = r(s)$.

En cada caso guardamos el posiblemente nuevo $\text{MBSC} = M_e$ y lo guardamos en $A[e]$.

Actualizamos k al número actual de puntos azules de MBSC.

Lema 2.DMBSC

Después de insertar un punto azul p , la estructura de datos es actualizada en $O((m + n) \log m)$ usando el algoritmo Insert-Blue-Update.

Demostración: Actualizar e insertar un punto de evento saliente en $\text{XEP}(e)$ toma $O(\log m)$. Dado que hay $O(m)$ de estos puntos en todo el diagrama, el total es $O(m \log m)$.

Iterar por las n aristas toma $O(n)$. Para cada arista, re-calcula $l(q_e^*)$ y se encuentra el nuevo q_e^* en tiempo $O(\log m)$. Se tiene $O(n \log m)$.

Encontrar a $r(s)$ toma $O(\log m)$.

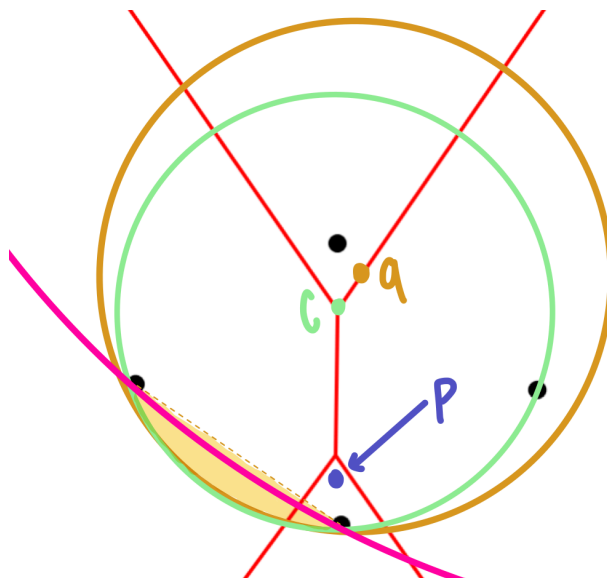
Sumando en total $O((m + n) \log m)$. ■

Eliminación de punto(Delete-Blue-Query(A, k, p))

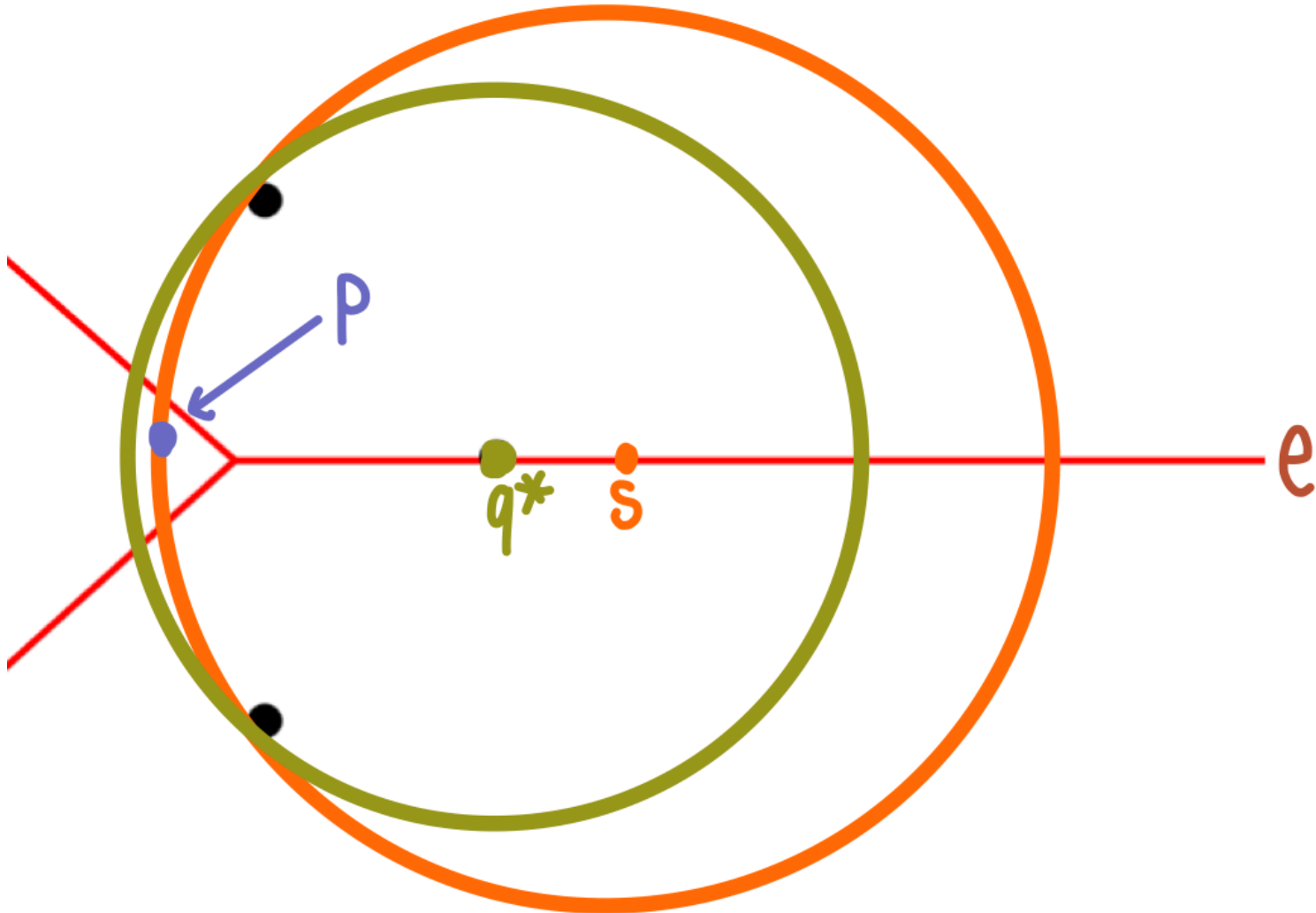
Si $p \in CH(R)$, entonces el MBSC actual sigue siendo el óptimo.

Si $p \notin CH(R)$, consideramos s el punto de evento generado por p (calculado en $O(1)$). Por cada $e \in FVD(R)$ tenemos 5 casos:

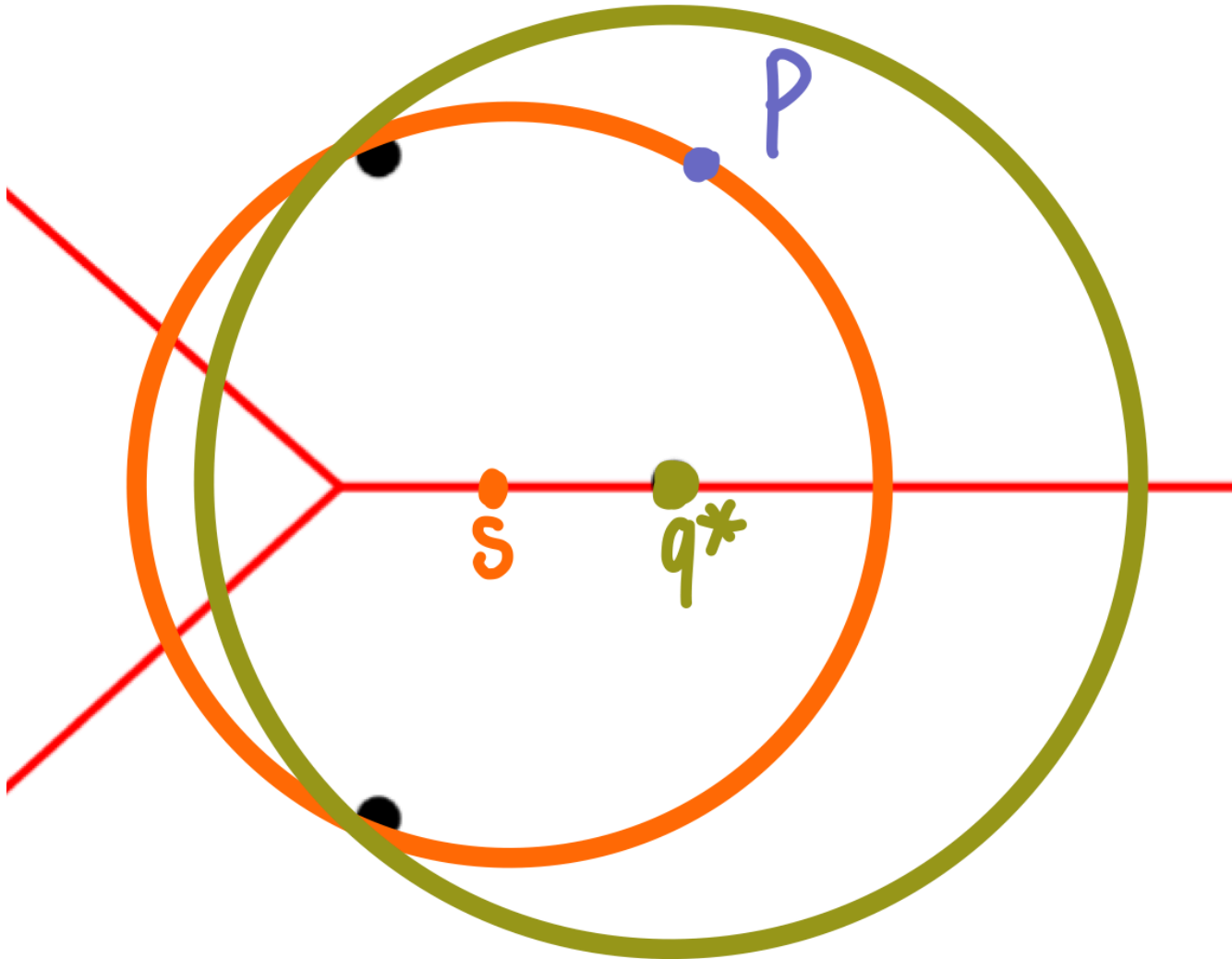
1. s no es un punto de evento en e : El MBSC actual es el óptimo. Si los círculos en e contiene a p , restamos un punto azul.



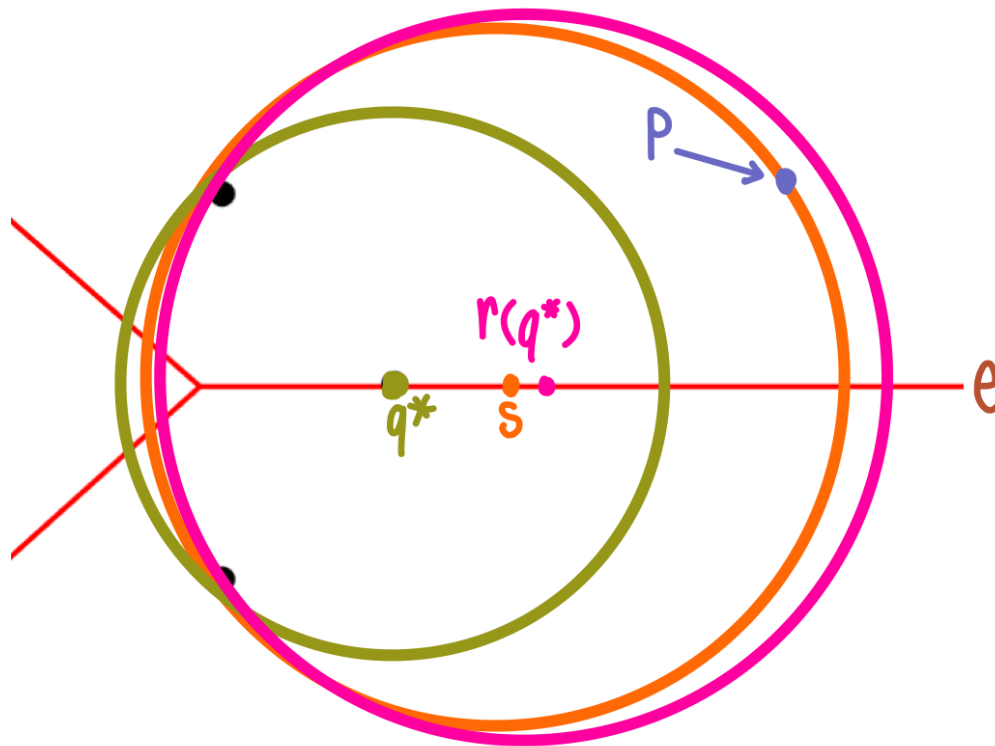
2. $q_e^* < s$ y s es un punto de evento saliente: $C(q_e^*)$ ya incluía a p , y ahora lo pierde (removimos a p). El MBSC actual se mantiene y restamos un punto azul.



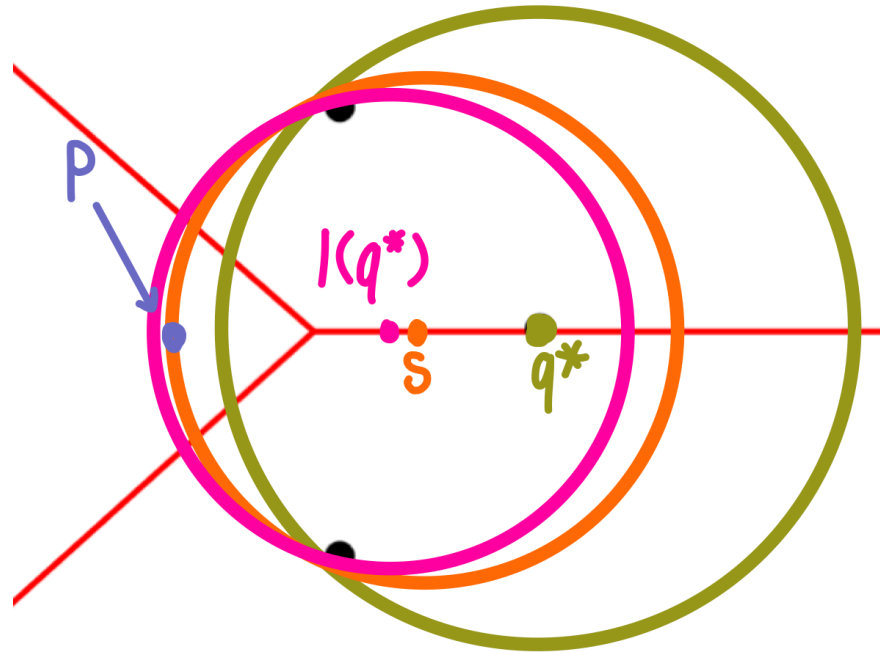
3. $s < q_e^*$ y s es un punto de evento entrante: $C(q_e^*)$ ya incluía a p , y ahora lo pierde (removimos a p). El MBSC actual se mantiene y restamos un punto azul.



4. $q_e^* < s$ y s es un punto de evento entrante: $C(q_e^*)$ no contiene a p , pero hay círculos a la derecha de él que si lo tienen, y por tanto reducen en uno su número de puntos azules.
- Si $s < r(s) \in \text{XEP}(e)$ existe, $C(s)$ es el nuevo MBSC con un punto azul menos. En otro caso, el MBSC actual se mantiene con los mismos puntos azules.



5. $s < q_e^*$ y s es un punto de evento saliente: $C(q_e^*)$ ya excluyó a p , pero hay círculos a la izquierda de él que si lo tienen, y por tanto reducen en uno su número de puntos azules.
- Si $l(q_e^*) < s$ existe, el MBSC nuevo es $C(l(q_e^*))$ con un punto azul menos. En otro caso, el MBSC actual se mantiene con los mismos puntos azules.



Después de procesar las n aristas, seleccionamos el MBSC tal que $m(\text{MBSC})$ y $\text{rad}(\text{MBSC})$ sean los menores.

Lema 3.DMBSC

Dado un punto p a ser eliminado, los MBSC son reportados en $O(n \log m)$ usando el algoritmo Delete-Blue-Query.

Demostración:

Busca en las n aristas del $\text{FVD}(R)$ en tiempo $O(n)$.

Por cada $e \in \text{FVD}(R)$, se calcula el punto de evento s de p y su tipo en tiempo $O(1)$.

Los casos 1), 2), 3), y 5) se resuelven en $O(1)$: Posición de s respecto a q_e^* y tipo de evento. Para 5) usamos $l(q_e^*)$.

Para 4), se busca en tiempo $O(\log m)$ en $XEP(e)$ a $r(s)$. Primero por número de puntos azules y después por radio. Como s es entrante, puede ocurrir en todas las aristas. Tenemos $O(n \log m)$.

Sumando en total $O(n \log m)$



Actualización de Eliminación (Delete-Blue-Update(A, k, p))

Para cada $e \in \text{FVD}(R)$, $s \in e$ el punto de evento correspondiente de p , procedemos según el caso ejecutado el *Delete-Blue-Query*:

1. Si p está contenido en los círculos de e , decrementamos m_q en 1 para todo q evento saliente en $\text{XEP}(e)$.
2. Para todo $q < s$ evento en $\text{XEP}(e)$, decrementamos m_q en 1, y lo relocalizamos en el árbol. Borramos s de $\text{XEP}(e)$.
3. Para todo $q > s$ evento en $\text{XEP}(e)$, decrementamos m_q en 1, y lo relocalizamos en el árbol. Re-calculamos $l(q_e^*)$.

4. Para todo $q > s$ evento en $\text{XEP}(e)$, decrementamos m_q en 1, y lo relocalizamos en el árbol. Si $r(s)$ estaba definido, $q_e^* = r(s)$.
5. Para todo $q < s$ evento en $\text{XEP}(e)$, decrementamos m_q en 1, y lo relocalizamos en el árbol. Si $l(q_e^*)$ estaba definido, $q_e^* = l(q_e^*)$ y recalculamos $l(q_e^*)$. Borramos s de $\text{XEP}(e)$.

En cada caso guardamos el posiblemente nuevo $\text{MBSC} = M_e$ y lo guardamos en $A[e]$.

Actualizamos k al número actual de puntos azules de MBSC .

Lema 4.DMBSC

Después de remover un punto azul p , la estructura de datos es actualizada en $O((m + n) \log m)$ usando el algoritmo Delete-Blue-Update.

Demostración:

Actualizar y eliminar un punto de evento saliente en $\text{XEP}(e)$ toma $O(\log m)$. Dado que hay $O(m)$ de estos puntos en todo el diagrama, el total es $O(m \log m)$.

Iterar por las n aristas toma $O(n)$. Para cada arista, re-calcula $l(q_e^*)$ y se encuentra el nuevo q_e^* en tiempo $O(\log m)$. Se tiene $O(n \log m)$.

Sumando en total $O((m + n) \log m)$



Complejidades para el Problema DMBSC

Teorema 5

Los puntos rojos y azules pueden ser preprocesados en $O(n \log n + mn \log m)$ en una estructura de datos de tamaño $O(m + n)$ que soporta inserciones y eliminaciones de puntos azules en $O((m + n) \log m)$ y puede ser usada para reportar la nueva solución en $O(n + \log m)$ al insertar un punto azul, y en $O(n \log m)$ al remover un punto azul.

Demostración:

Procesamos los puntos iniciales con el algoritmo del MBSC en tiempo $O(n \log n + mn \log m)$ y en $O(m + n)$ espacio. El tamaño es asintóticamente igual dado que XEP es $O(m)$

(eventos salientes). Las complejidades de inserciones, eliminaciones, reportaje, y actualizaciones se siguen de los lemas pasados. ■

Método Logarítmico de Inserciones y Eliminaciones de Puntos Azules

Inserciones Logarítmicas de Puntos Azules

Eficiente solo para inserciones de puntos azules, pero usa más memoria y toma más tiempo actualizar la estructura de datos.

Estructura de Datos

Un árbol balanceado binario de búsqueda T con $O(m)$ nodos.

- Indexado por la cantidad i de puntos azules dentro de un círculo.
- Cada nodo T^i es un puntero a un árbol binario de búsqueda balanceado que contiene círculos tales que $m(C) = i$.
- Hojas: Guardan los círculos ordenados por su radio, y la arista en la que se encuentra.

- Nodo interno: Guarda la intersección de los círculos que son sus descendientes.

Observación 1.LI

Cada T^i es una extensión de Offline Ball Exclusion Search Data Structure (OLBES).

Observación 2.LI

En total hay $O(mn)$ círculos en T (n aristas y m puntos de eventos).

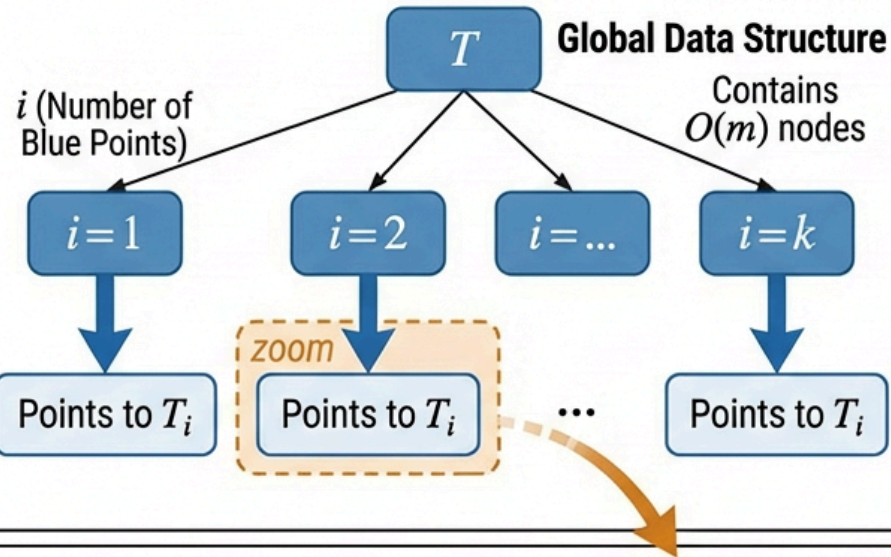
Observación 3.LI

Cada T^i tiene un máximo de $O(mn)$ círculos.

Observación 4.LI

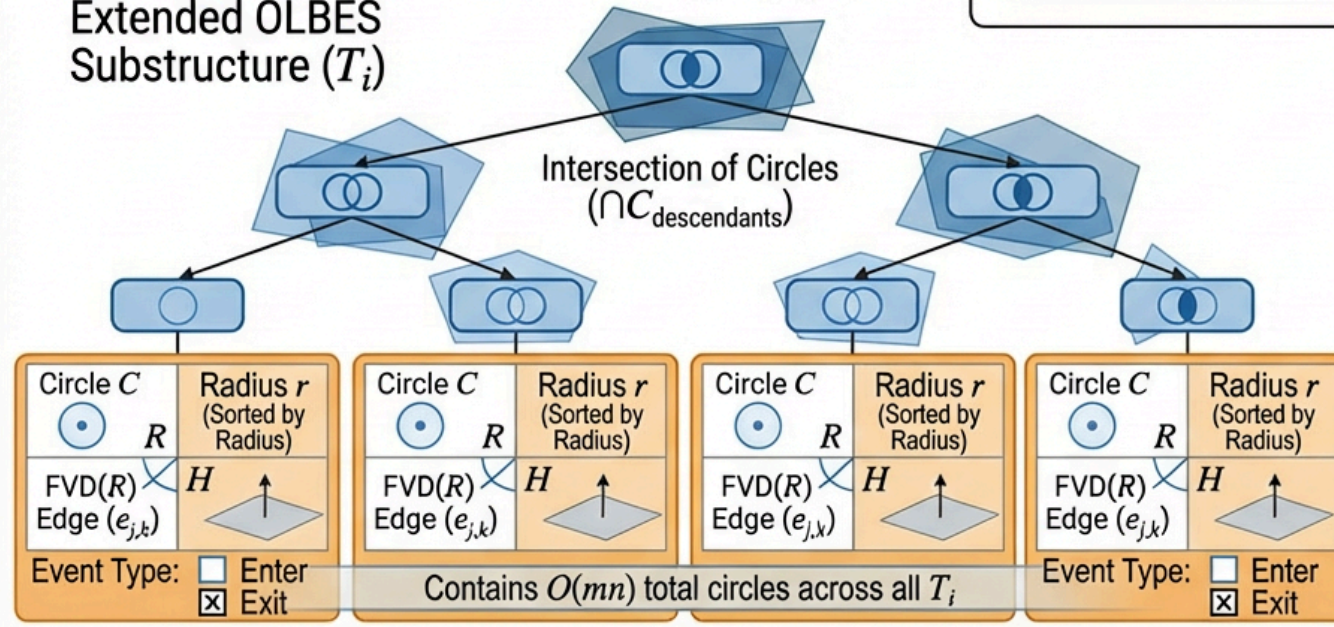
Al haber $O(\log mn)$ niveles, y dado que los $O(mn)$ círculos contribuyen a la intersección en cada nivel, el total de espacio usado es $O(mn \log(mn))$.

A Global Hierarchy (Main Tree T)



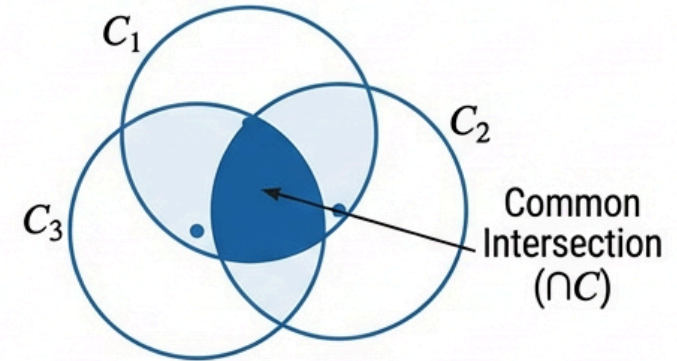
B Detailed Extended OLBES Structure (T_i):

Extended OLBES Substructure (T_i)



C Geometric Concept (The "Why" of OLBES)

Geometric Aggregation: Intersection of Circles



Query Point $p \in \cap C \Leftrightarrow p$ is enclosed by ALL descendant circles in the subtree.

Global Tree Nodes:

Blue Rectangles (A);

Subtree Nodes:

Blue Rectangles (B);

Leaf Data Fields:

Orange Compartments (B);

Geometric Intersections:

Heavily Shaded Blue (C).

Insertar Punto Azul (Insert-Blue-Query-Logtime(T, p))

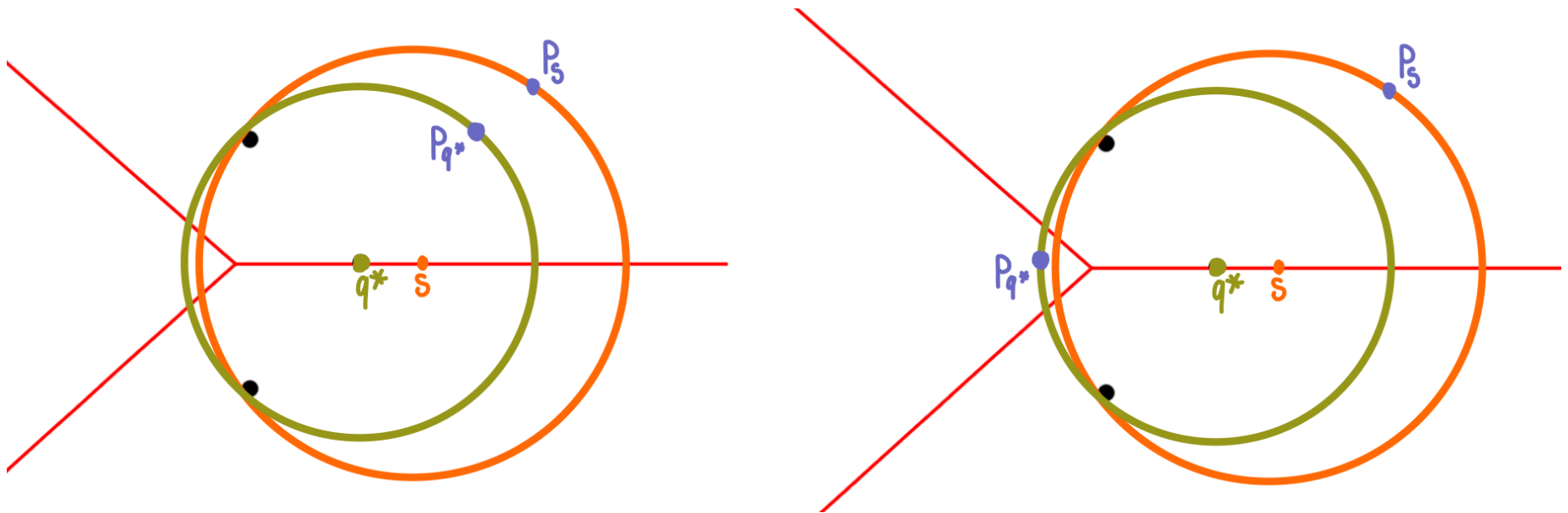
Consideramos k el valor mínimo de puntos azules del MBSC actual.

Hacemos una query en el nodo T^k con p :

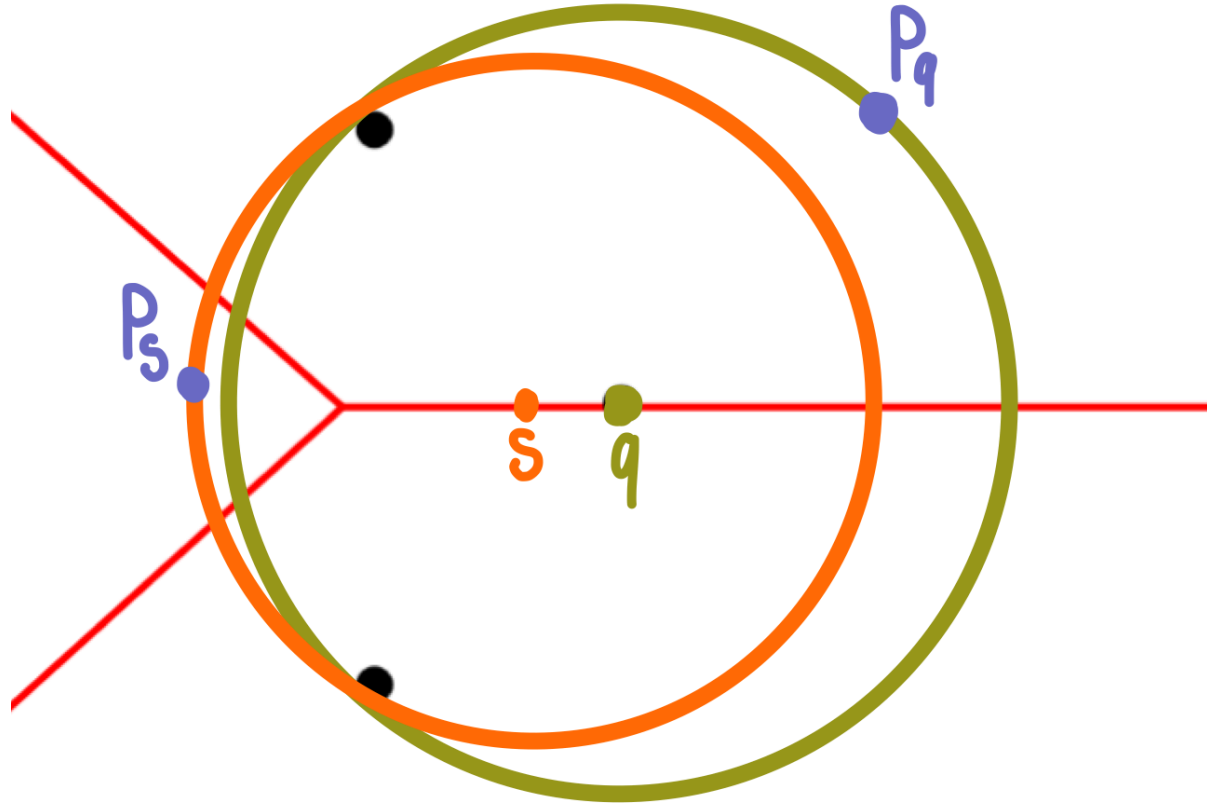
1. Éxito: Regresa $C(q)$, sobre la arista e , tal que no contiene a p y tiene radio mínimo.

Consideramos el tipo de evento de s :

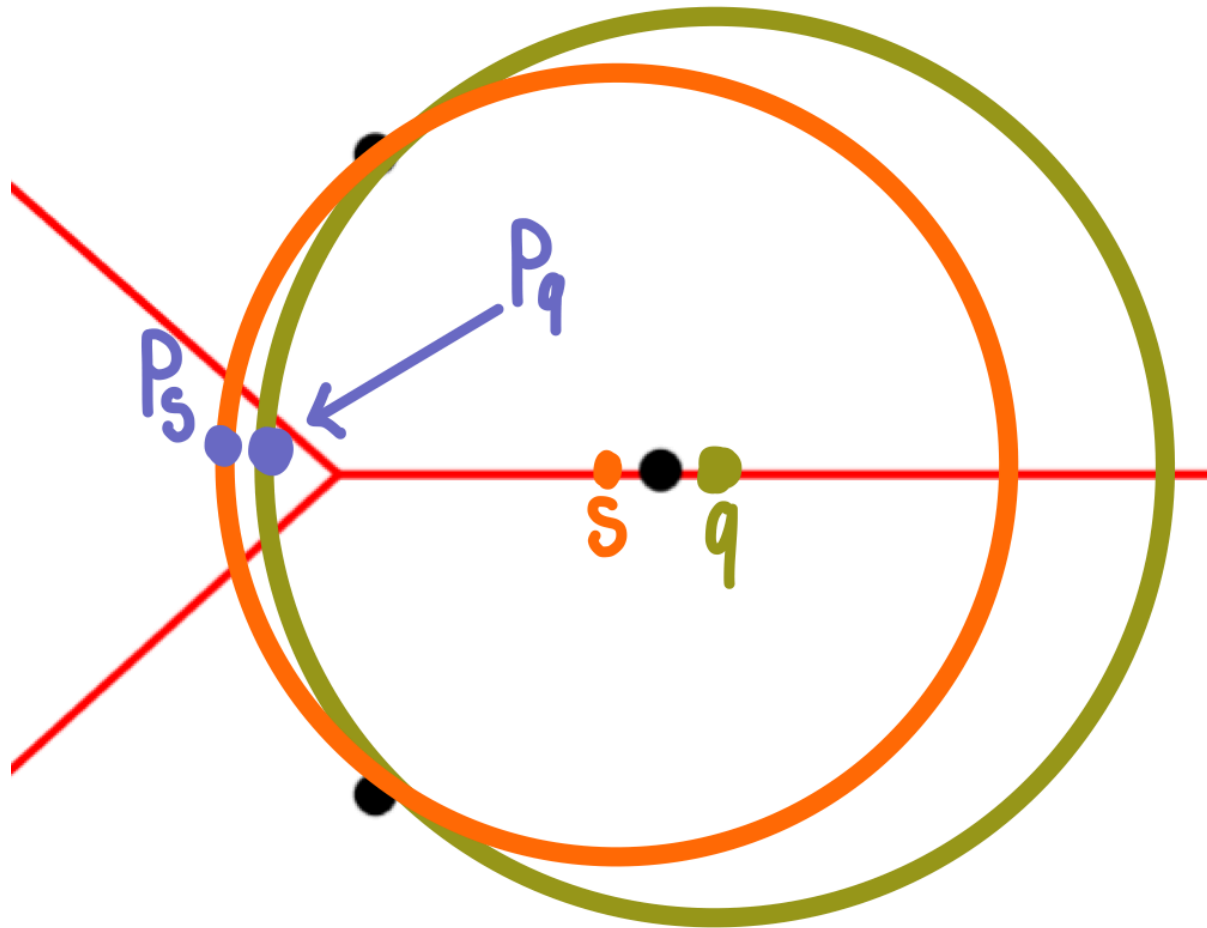
- Entrante: Necesariamente $q < s$ (no incluye a p). La inserción de p solo afecta a $C(t)$ tal que $t \geq s$. $C(q)$ sigue siendo el óptimo.



- Saliente: Consideramos el tipo de evento de q :
 - Entrante: Necesariamente $s < q$. Todo $C(t)$ con $t < s$ ahora tiene al menos $k + 2$ puntos. Todo $C(t')$ con $t' > s$ tiene al menos k puntos. $C(s)$ es tal que excluye a p y p_q . El óptimo es $C(s)$.

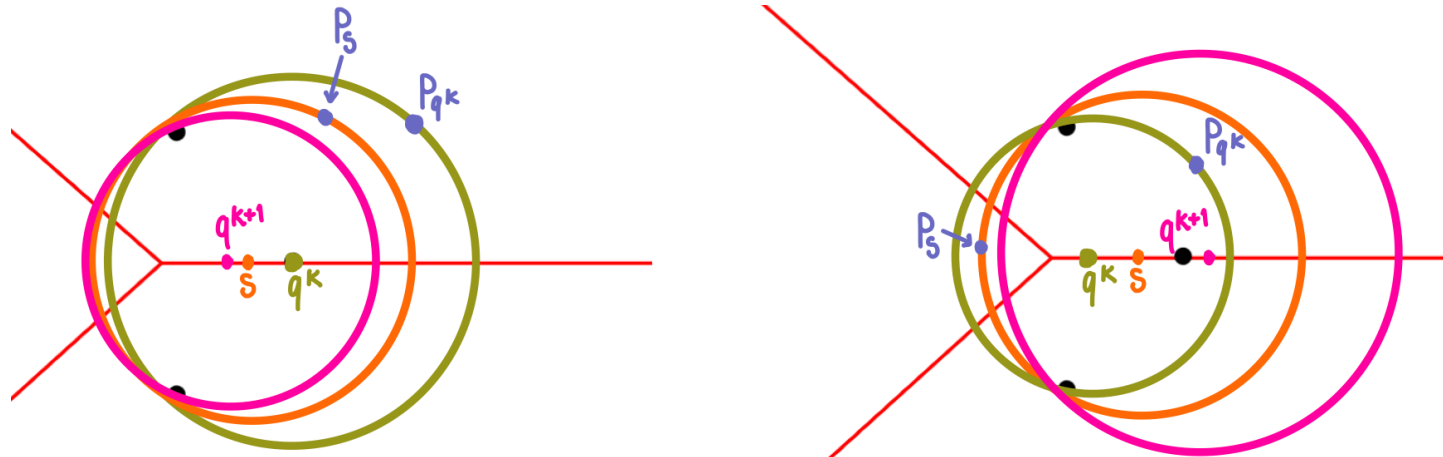


- Saliente: Necesariamente $s < q$. Todo $C(t)$ con $t < s$ tiene al menos $k + 2$ puntos. Todo $C(t')$ con $t' > s$ tiene al menos k . $C(q)$ es tal que excluye a p y a p_q . El óptimo es $C(q)$.



2. No éxito: Todos los círculos con k puntos ahora tienen $k + 1$ (contienen a p).

- Consideramos al mejor de $T^k: C(q^k)$ tal que $m_{q^k} = k$ y tiene radio mínimo.
- Hacemos una query en el nodo T^{k+1} con p :
 - Éxito: Bajo los mismos casos de éxito del escenario 1 encontramos $C(q^{k+1})$ tal que no contiene a p . Regresamos $\min\text{-radius}\{C(q^k), C(q^{k+1})\}$.



- No éxito: Todos $C(q^{k+1})$ tienen $m_{q^{k+1}} = k + 2$. El óptimo es $C(q^k)$.

Actualización de Inserción (Insert-Blue-Update-Logtime(T , p))

1. Sean l y k el máximo y mínimo número de puntos azules en T .
2. Para cada i desde l hacia k :
 - 2.1. Para cada $C \in T^i$ tal que $p \in C$, movemos C de T^i hacia T^{i+1}
 - 2.2. Si el nodo T^i queda vacío, lo borramos.
3. Para cada arista $e \in \text{FVD}(R)$:
 - 3.1. Consideramos $s \in e$ el punto de evento definido por p , y $C(s)$.
 - 3.2. Guardamos el tipo de evento de s .
 - 3.3. Contamos el número k' de puntos azules en C .
 - 3.4. Añadimos C a $T^{k'}$.
4. Para cada i desde k hacia l (bottom-up):

4.1 Actualizamos el árbol correspondiente a T^i : reordenamos por radio y calculamos todas las intersecciones hasta la raíz.

```

def insert_blue_update_logtime(T, p):
    l = max_bp() // 0(1)
    k = min_bp() // 0(1)
    for i in range(l, k): // 0(m)
        circles = circles_that_contain_point(T[i], p) // 0(log mn)
        if not T[i + 1]: create_node(i + 1) // 0(log m)
        for c in circles: // 0(mn)
            move_circle_from_to(c, T[i], T[i + 1]) // 0(log mn)
        if T[i] is empty: delete_node(i) // 0(log m)

    for e in FVD(R): // 0(n)
        s = event_point(p) // 0(1)
        c = circle_from_event_point(s) // 0(1)
        evt = event_point_type(s) // 0(1)
        bp = get_bp(c) // 0(1)

        if not T[bp]: create_node(bp) // 0(log m)
        insert(c, s, evt) // 0(log mn)

    for i in range(k, l): // 0(m)
        update_tree(i) // 0(mn log mn)

```


Lema 6.DMBSC

Con un preprocesado de $O(mn \log(mn))$ en tiempo y espacio, la inserción de un punto azul se reporta en $O(\log(mn))$ y en $O(mn \log(mn))$ se actualiza la estructura de datos con Insert-Blue-Query-Logtime e Insert-Blue-Update-Logtime, respectivamente.

Demostración:

- La estructura OLBES nos permite hacer una query en un conjunto de $O(c)$ círculos en $O(\log c)$ con un preprocesado de $O(c \log c)$ en tiempo y espacio. Como cada T^i puede tener $O(mn)$ círculos, $c = mn$.
- Para reportar:

Lo hacemos en $O(\log mn)$ después de un preprocesado de $O(mn \log(mn))$ (comparaciones y solo una búsqueda en el árbol).

- Para la actualización:
 - ▶ Paso 2: podemos tener $O(mn)$ círculos con p en todo T , distribuidos por los T^i 's. Para cada T^i consideramos los n_i círculos correspondientes. Toma $O(\log n_{i+1})$ re-insertarlo en T^{i+1} (ganó un punto azul). Al tener $O(mn)$ círculos, toma $O(mn \log(mn))$ (actualizamos todo).
 - ▶ Paso 3: iteramos en las n aristas de $FVD(r)$ y creamos n nuevos círculos y contamos sus puntos azules. Para n círculos y m puntos posibles, $O(mn)$. Añadir los n nuevos círculos toma $O(n \log(mn))$

- Paso 4: actualizamos cada T^i . Se deben reconstruir las intersecciones para los $O(\log mn)$ niveles. Toma $O(mn \log(mn))$.

El tiempo total es $O(mn \log(mn))$. ■

Eliminaciones Logarítmicas de Puntos Azules (Delete-Blue-Query-Logtime(U, p))

Eficiente solo para eliminaciones de puntos azules, pero usa más memoria y toma más tiempo actualizar la estructura de datos.

Estructura de Datos

Un árbol balanceado binario de búsqueda U con $O(m)$ nodos.

- Indexado por la cantidad i de puntos azules dentro de un círculo.
- Cada nodo U^i es un puntero a un árbol binario de búsqueda balanceado que contiene círculos tales que $m(C) = i$.
- Hojas: Guardan los círculos ordenados por su radio, y la arista en la que se encuentra.

- Nodo interno: Guarda la intersección de los semi-planos asociados a los círculos que son sus descendientes.

Observación 1.DI

Cada U^i es una extensión de Offline Ball Inclusion Search Data Structure (OLBIS).

Observación 2.LI

En total hay $O(mn)$ círculos en U (n aristas y m puntos de eventos).

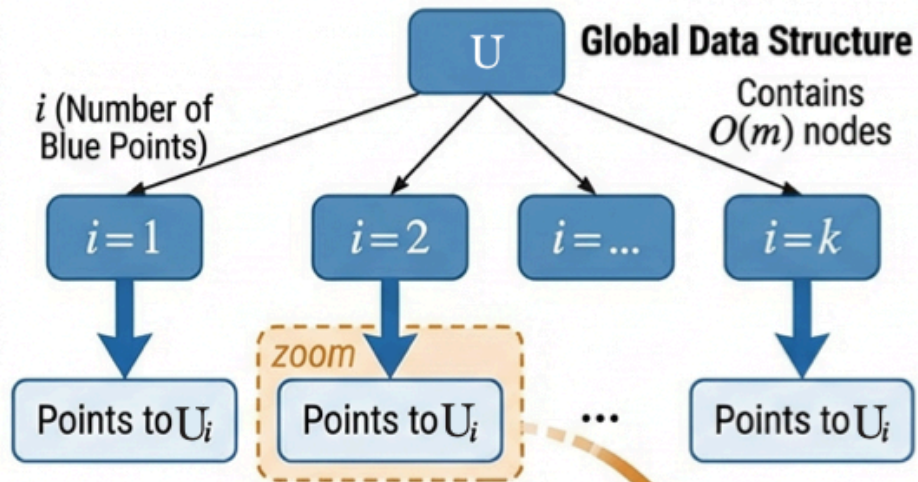
Observación 3.LI

Cada U^i tiene un máximo de $O(mn)$ círculos.

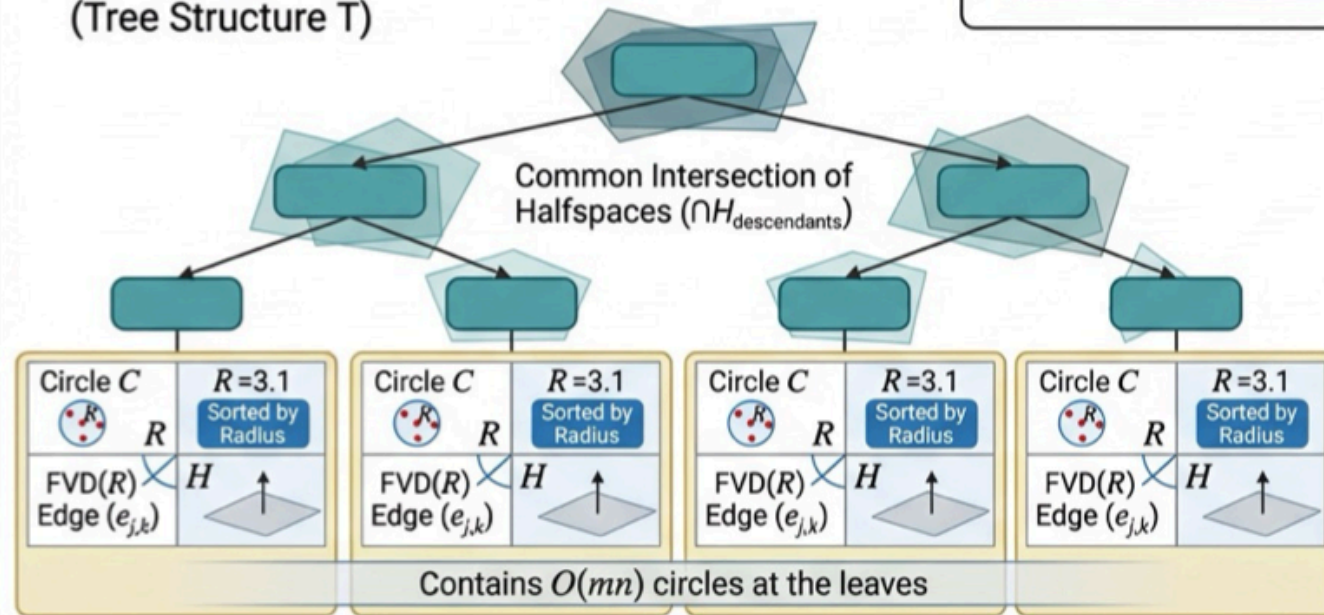
Observación 4.LI

Al haber $O(\log mn)$ niveles, y dado que los $O(mn)$ círculos contribuyen a la intersección en cada nivel, el total de espacio usado es $O(mn \log(mn))$.

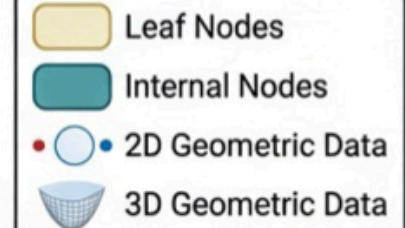
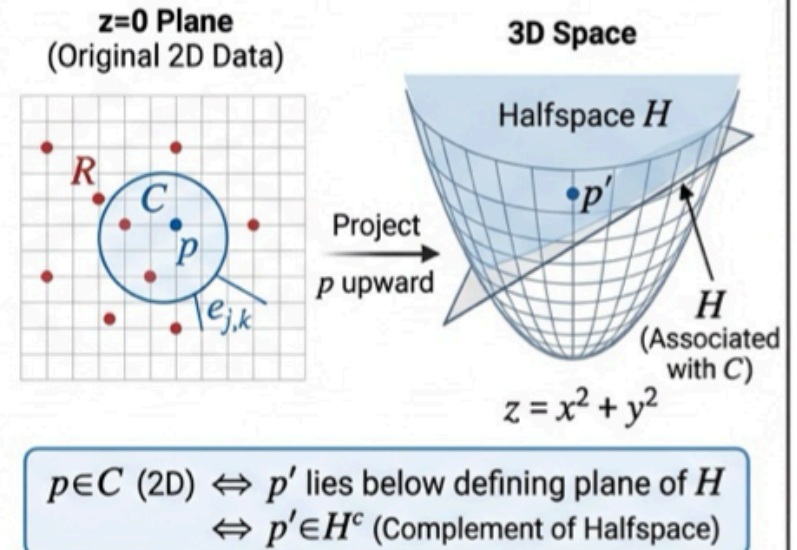
A Global Hierarchy (Main Tree U)



B OLBIS Substructure Detail U_i (Tree Structure T)



C Transformation: 2D Circle Inclusion to 3D Halfspaces



Eliminar punto (Delete-Blue-Query-Logtime(U, p))

Consideramos k el valor mínimo de puntos azules del MBSC actual.

Hacemos una query en el nodo U^k con p :

1. Éxito: Regresa $C(q)$ tal que contiene a p y tiene radio mínimo. $C(q)$ se mantiene como el óptimo, y disminuye en uno su conteo.
2. No éxito: Todos circulo con k puntos azules no contienen a p y, entonces, no reducen puntos.
 - Consideramos al mejor de U^k : $C(q^k)$ tal que $m_{q^k} = k$ y tiene radio mínimo.
 - Hacemos una query en el nodo U^{k+1} con p :

- ▶ Éxito: Devuelve $C(q^{k+1})$ tal que contiene a p . Regresamos $\text{min-radius}\{C(q^k), C(q^{k+1})\}$.
- ▶ No éxito: p no estaba en ningún círculo de U^{k+1} . El óptimo es $C(q^k)$.

Actualización de Eliminación (Delete-Blue-Update-Logtime(U , p))

1. Consideramos l y k el máximo y mínimo número de puntos azules en cualquier círculo, respectivamente, de U .
2. Para cada i desde k hacia l (bottom-up):
 - 2.1. Para cada círculo $C \in U^i$ tal que $p \in C$, movemos C de U^i hacia U^{i-1} (lo creamos si es necesario).
 - 2.2. Eliminamos todos los círculos centrados en el evento asociado a p .

2.3. Si el nodo U^i queda vacío, lo borramos.

3. Para cada i desde k hacia l (bottom-up):

3.1 Actualizamos el árbol correspondiente a U^i : reordenamos por radio y calculamos todas las intersecciones hasta la raíz.

```

def delete_blue_update_logtime(U, p):
    l = max_bp() // 0(1)
    k = min_bp() // 0(1)
    for i in range(k, l): // 0(m)
        circles = circles_that_contain_point(U[i], p) // 0(log mn)
        if not U[i - 1]:
            create_node(i - 1) // 0(log m)

        for c in circles: // 0(mn)
            if event_point(c) == event_point(p):
                delete_circle(c) // 0(log mn)
            else:
                move_circle_from_to(c, U[i], U[i - 1]) // 0(log mn)

        if U[i] is empty:
            delete_node(i) // 0(log m)

    for i in range(k, l): // 0(m)
        update_tree(i) // 0(mn log mn)

```

Lema 7.DMBSC

Con un preprocesado de $O(mn \log(mn))$ en tiempo y espacio, la eliminación de un punto azul se reporta en $O(\log^2(mn))$ y en $O(mn \log(mn))$ se actualiza la estructura de datos con Delete-Blue-Query-Logtime e Delete-Blue-Update-Logtime, respectivamente.

Demostración:

- La estructura OLBIS nos permite hacer una query en un conjunto de $O(c)$ círculos en $O(\log^2 c)$ con un preprocesado de $O(c \log c)$ en tiempo y espacio. Como cada U^i puede tener $O(mn)$ círculos, $c = mn$.
- Para reportar:

Lo hacemos en $O(\log^2 mn)$ después de un preprocesado de $O(mn \log(mn))$ (comparaciones y solo una búsqueda en el árbol).

- Para la actualización:
 - ▶ Paso 2: Podemos tener $O(mn)$ círculos con p en todo U , distribuidos por los U^i 's. Para cada U^i consideramos los n_i círculos correspondientes. Toma $O(\log n_{i-1})$ re insertarlo en U^{i-1} (perdió un punto azul). Al tener $O(mn)$ círculos, toma $O(mn \log(mn))$ (actualizamos todo). Eliminar n círculos (los relacionados con p) toma $O(n \log(mn))$.
- Paso 3: Actualizamos cada U^i . Se deben reconstruir las intersecciones para los $O(\log mn)$ niveles. Toma $O(mn \log(mn))$.

El tiempo total es $O(mn \log(mn))$. ■

Referencias

- [1] B. Armaselu and O. Daescu, “Dynamic minimum bichromatic separating circle,” *Theoretical Computer Science*, vol. 774, pp. 133–142, 2019, doi: <https://doi.org/10.1016/j.tcs.2016.11.036>.
- [2] S. Bitner, Y. Cheung, and O. Daescu, “Minimum Separating Circle for Bichromatic Points in the Plane,” in *2010 International Symposium on Voronoi Diagrams in Science and Engineering*, 2010, pp. 50–55. doi: [10.1109/ISVD.2010.14](https://doi.org/10.1109/ISVD.2010.14).